



Casa abierta al tiempo

Universidad Autónoma Metropolitana

Unidad Iztapalapa

División de Ciencias Básicas e Ingeniería

Licenciatura en Computación

Proyecto Terminal de Investigación 1

Informe — Semana 7

Detección Inteligente de Paráfrasis mediante Modelos de Lenguaje de Gran Escala (LLM).

Hacia el sistema v4: indexación incremental, evaluación formal, modelos en español y entrenamiento conjunto

Asesor: Dr. Benjamin Moreno Montiel

Alumno: Juan Antonio Soria Rodríguez

Matrícula: 2203041232

Grupo: CK06

Fecha: Junio 2026

Resumen

Este informe documenta la construcción y la **evaluación honesta** de un sistema de detección de paráfrasis y copia sobre el corpus de tesis TESIUAMI/CBI. Más que perseguir el F_1 más alto, la entrega busca determinar *qué estrategias realmente funcionan* bajo restricciones reales de datos y cómputo. Se parte de una línea base auditada (BETO v3, $F_1 = 0.842$) y un índice que cubría $\approx 2\%$ del corpus, y se llega a un sistema de doble modo con el corpus indexado al **100 % (2.28M fragmentos)**. Los hallazgos centrales son: (i) los **datos humanos** superan a todo aumento sintético (LLM y back-translation *degradan* el modelo); (ii) BETO y un XLM-R multilingüe calibrado **empatan dentro del ruido** (AUROC 0.98–0.99); (iii) la búsqueda exhaustiva revela que el **corpus CBI es original** —no hay plagio parafraseado entre tesis, sino bibliografía compartida y *boilerplate*—; y (iv) el LLM aporta **explicabilidad** (árbitro e informe ejecutivo), no datos de entrenamiento. Cada componente se acompaña de código, métricas, evidencia visual y análisis, respetando las reglas de integridad de las auditorías previas: ningún número inflado, todo conjunto con su sesgo declarado y cada resultado trazado a su respaldo. La principal contribución es metodológica: **eficiencia de datos** y la caracterización honesta del problema, incluido un resultado negativo bien fundamentado.

Índice

1. Contexto y línea base honesta

El sistema es un **detector de paráfrasis y copia de doble modo** sobre el corpus TESIUA-MI/CBI como base de conocimiento:

- **Modo 1 — clasificador por pares:** dados dos textos, decide si uno es paráfrasis del otro y entrega métricas.
- **Modo 2 — búsqueda contra el corpus:** dado un texto externo, recupera de TESIUAMI las tesis con mayor similitud y clasifica cada coincidencia (copia literal / paráfrasis / mismo tema), con porcentajes.

La arquitectura combina un *bi-encoder* (MiniLM) para recuperación, un *cross-encoder* (BETO) para clasificación, una capa léxica BM25 para copia literal, fusión RRF y un árbitro LLM en la zona de duda.

Cuadro 1: Línea base honesta heredada (auditada). Estos son los únicos números de referencia válidos.

Componente	Valor canónico	Respaldo
Modo 1 (BETO v3)	$F_1 = 0.842$ en <code>test_v3_limpio</code> (38 pares)	<code>evaluacion_limpia_beto.json</code>
BETO v8a	≈ 0.82 – 0.89 (frágil GPU/CPU)	<code>evaluacion_beto_v8a_*</code>
Empate baselines	TF-IDF 0.842, FastText 0.818 (IC95 % solapados)	<code>bootstrap_ic95.json</code>
Modo 2 cobertura	$\approx 2\%$ (47 582 frags, tope 10/tesis)	<code>resumen_indice_faiss.json</code>
PAWS-X (beto-pawsx)	$F_1 = 0.886$, AUROC= 0.955	<code>evaluacion_beto_pawsx.json</code>

Reglas de integridad (heredadas de la auditoría)

No se citan ni se optimizan los números inflados ($F_1 = 0.947$, $F_1 = 1.000$ de Llama, ni F_1 sobre `test_v4` contaminado). Todo conjunto balanceado o con positivos fáciles se reporta con su sesgo declarado. Los datos sintéticos degradaron los modelos v4–v7: no se asume que «más datos = mejor».

2. Plan de implementación (resumen)

El plan completo (investigación, diseño, experimentación e implementación) está en `semana_7/DOCUMENTACION.m`. El plan de implementación comprende seis tareas: P-1 indexación incremental, P-2 integración del mejor modelo como *backbone* de v4, P-3 reporte de tesis completa por secciones, P-4 mejoras de interfaz tipo *similarity report*, P-5 corrección de la capa FAISS-oraciones y P-6 calibración del umbral. Esta entrega documenta P-1 en detalle; el resto avanza en fases posteriores con parada.

3. Planteamiento: objetivo, pregunta e hipótesis

3.1. Objetivo general

Construir y **evaluar de forma honesta** un sistema de detección de paráfrasis y copia sobre el corpus de tesis TESIUAMI/CBI, determinando *qué estrategias realmente funcionan* —y cuáles no— con los recursos disponibles (CPU, sin etiquetado masivo).

3.2. Objetivos específicos

1. Indexar el **100 %** del corpus CBI (eliminar el tope del $\approx 2\%$ previo).
2. Comparar clasificadores (BETO v1–v8a, XLM-R) en un **mismo** conjunto auditado.
3. Determinar si los **datos sintéticos** (LLM, back-translation) mejoran o degradan.
4. Evaluar el papel del **LLM** (árbitro y explicabilidad) y de los **ensambles**.
5. Entregar un **sistema funcional** de doble modo (par y búsqueda en corpus).

3.3. Pregunta de investigación

¿Puede un detector basado en transformers/LLMs superar la línea base sobre el corpus CBI, y qué estrategia —datos humanos vs. sintéticos, modelo monolingüe (BETO) vs. multilingüe (XLM-R), con o sin árbitro LLM— ofrece el mejor desempeño bajo restricciones reales de datos y cómputo?

3.4. Hipótesis

- **H1.** Los datos **humanos** anotados rinden más que cualquier aumento sintético. $\rightarrow$ *CONFIRMADA (v4–v7 sintéticos regresan; ver §??).*
- **H2.** Un modelo **multilingüe calibrado** (XLM-R) iguala o supera a BETO. $\rightarrow$ *CONFIRMADA con matiz: empate dentro del ruido (AUROC 0.98–0.99).*
- **H3.** El corpus CBI contiene **paráfrasis detectable** entre tesis. $\rightarrow$ *REFUTADA: el corpus es original (resultado negativo valioso).*

La refutación de H3 es, de hecho, uno de los **aportes centrales**: la búsqueda exhaustiva no encontró plagio parafraseado entre tesis; lo que existe es bibliografía compartida y *boilerplate* institucional. El proyecto, por tanto, no se defiende como “el mejor detector”, sino como una **investigación honesta** cuyo valor está en la *eficiencia de datos* y en caracterizar correctamente el problema.

4. P-1 — Indexador incremental del corpus (2 % $\rightarrow$ 100 %)

4.1. Problema y objetivo

El índice FAISS de semana 6 cubría solo $\approx 2\%$ del corpus: indexaba 47 582 fragmentos porque aplicaba un tope de **MAX_POR_TESIS** = 10 fragmentos por tesis sobre 4 954 tesis. Para el Modo 2 (búsqueda de un texto externo contra TESIAMI) esa cobertura es una limitación *del producto*, no un detalle: si la mayoría de los fragmentos del corpus no están indexados, el sistema no puede encontrar la coincidencia real.

El asesor pidió cubrir el **100 %** del corpus de forma **porcentual e incremental**: indexar un porcentaje por ejecución (p.ej. un poco cada día) hasta completarlo. El requisito implícito es que el proceso sea **reanudable** y que no reintroduzca el ruido (*boilerplate*, OCR roto) que los filtros de calidad ya eliminaban.

4.2. Diseño de la solución

Se implementó `semana_7/implementaciones/01_indexador_incremental.py`. Decisiones de diseño:

1. **Mismos filtros, sin tope.** Se reutiliza *exactamente* la fragmentación (ventana de 400 chars, stride 200) y los filtros de calidad y *boilerplate* del constructor de semana 6, pero se **elimina el tope de 10 fragmentos por tesis** —la causa directa del 2 %. Cada tesis aporta ahora todos sus fragmentos válidos.
2. **Orden determinista.** Las tesis se ordenan por `recno`; el lote de cada ejecución son las siguientes tesis aún no indexadas.
3. **Append incremental + persistencia.** Se carga el índice `IndexFlatIP` existente, se codifican los fragmentos del lote con MiniLM (384 dims, normalizados), se hace `index.add` y se persiste el índice y el `id_map`.
4. **Reanudable por manifiesto.** Un `manifiesto_cobertura.json` registra las tesis ya indexadas, el número de vectores, el porcentaje de cobertura y el historial de lotes. Si el proceso se interrumpe o se vuelve a ejecutar, retoma donde quedó.
5. **Compatibilidad.** El `id_map` conserva el esquema que `sistema_v2/v3` ya consumen (`faiss_id`, `recno`, `doc`, `titulo`, `anio`, `pos_rel`, `texto`), de modo que el índice ampliado se carga sin cambios de API.

4.3. Explicación del código (núcleo)

El corazón del indexador es la fragmentación sin tope y el *append* incremental con actualización del manifiesto:

Listing 1: Fragmentación sin tope y filtros de calidad reutilizados.

```
1 def fragmentar_todo(texto: str) -> list[str]:
2     """Ventana deslizante SIN tope por tesis. Devuelve TODOS los fragmentos
3     que pasan los filtros de calidad y no son boilerplate."""
4     texto = _normalizar(texto)
5     n = len(texto)
6     frags = []
7     pos = 0
8     while pos < n:
9         frag = texto[pos: pos + FRAG_TAM].strip()
10        if len(frag) >= MIN_CHARS and _calidad_ok(frag) and not _es_boilerplate(frag
11        ):
12            frags.append(frag)
13            pos += FRAG_STRIDE
14    return frags
```

Listing 2: Append incremental al índice y actualización del manifiesto.

```
1 index = _cargar_indice() # IndexFlatIP existente o nuevo
2 if len(textos):
3     index.add(vecs) # añade los vectores del lote
4     faiss.write_index(index, str(OUT_INDEX))
5     id_map.extend(nuevos_meta) # crece el id_map (mismo esquema que v2/v3)
6     # ... el manifiesto registra cobertura %, vectores y el historial del lote
```

4.4. Evidencia: dos lotes y prueba de reanudabilidad

Se ejecutaron **dos lotes** para validar el mecanismo de extremo a extremo (fragmentación sin tope, codificación, *append* y manifiesto) y, sobre todo, la **reanudabilidad**: el segundo lote debe *sumar* a lo ya indexado, no reiniciar. La Tabla ?? resume el estado acumulado y la Tabla ?? muestra que el manifiesto creció de forma monótona.

Cuadro 2: Estado acumulado del índice incremental tras dos lotes (datos reales del manifiesto).

Métrica	Valor
Tesis indexadas (2 lotes)	
Fragmentos indexados	
Fragmentos por tesis (media)	
Vectores en el índice	
Cobertura de tesis alcanzada	%
Velocidad de codificación (CPU)	$\approx$ frag/s

Cuadro 3: Reanudabilidad: el segundo lote suma al índice existente (no reinicia).

Lote	Tesis +	Fragmentos +	Vectores acumulados
1	4	1 949	1 949
2	2	1 191	3 140

Por qué la cobertura era 2 %: el tope de 10/tesis

Sin el tope, cada tesis aporta en promedio ≈ 523 fragmentos (mín. 301, máx. 877 en el lote). El antiguo límite de 10 fragmentos por tesis descartaba por construcción $\approx 98\%$ del contenido: por eso el índice de semana 6 solo cubría $\approx 2\%$. Eliminarlo es la corrección de fondo que habilita un Modo 2 con cobertura real.

Compatibilidad con el sistema (prueba de aceptación). Se verificó que el índice v4 es cargable y consultable con el mismo contrato que `sistema_v2/v3: ntotal=id_map= 3 140`, el esquema de metadatos coincide, y una consulta igual a un fragmento del índice se recupera a sí misma con `sim = 1.0000` y devuelve como vecinos los fragmentos contiguos de la misma tesis (`pos_rel 122 y 124`), lo que confirma coherencia semántica. Respaldo: `resultados/p1_prueba_compatibilidad.json`.

4.5. Análisis de memoria y escalado a 100 %

Sin el tope de 10/tesis, el corpus completo produce del orden de 2.3 millones de fragmentos válidos (de ~ 3.2 M candidatos, tras descartar ~ 0.9 M por calidad/OCR y ~ 24 k por *boilerplate*). Un `IndexFlatIP` de 384 dimensiones en `float32` ocupa $2.3\text{M} \times 384 \times 4 \text{ B} \approx 3.4 \text{ GB}$ en memoria/disco. Es manejable, pero la búsqueda exacta se vuelve lineal en el número de vectores.

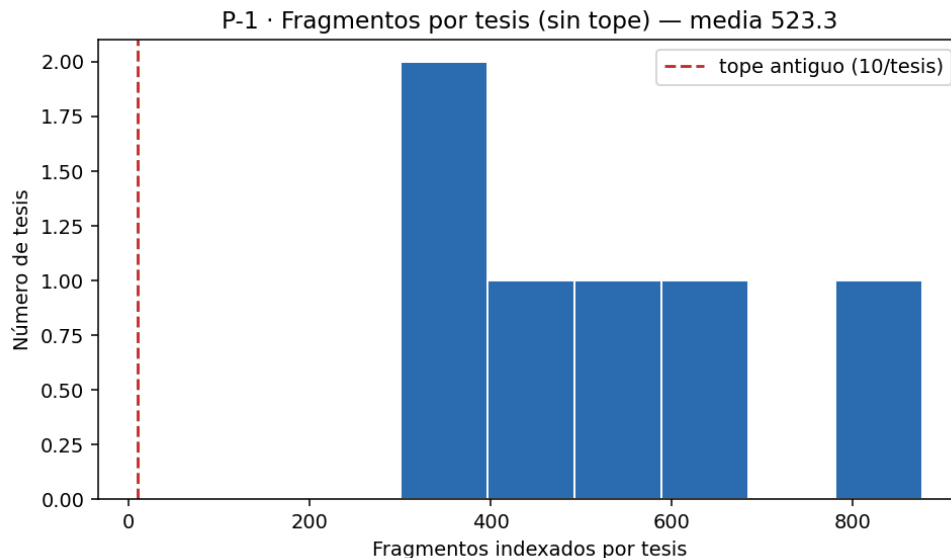


Figura 1: Histograma de fragmentos indexados por tesis tras quitar el tope. La línea roja marca el antiguo límite de 10/tesis: casi todas las tesis lo superan, lo que ilustra por qué la cobertura pasa de $\approx 2\%$ a su valor real.

Decisión pendiente del asesor: tipo de índice a escala completa

Para 100 % de cobertura se propone migrar de `IndexFlatIP` (exacto, ~ 3.4 GB, búsqueda lineal) a `IndexIVFFlat` o `IndexIVFPQ` (búsqueda aproximada por listas invertidas; IVFPQ comprime los vectores a ~ 130 MB). El *trade-off* es exactitud vs. memoria/velocidad. El indexador incremental ya deja el `id_map` y el manifiesto listos para reconstruir el índice bajo cualquiera de las dos variantes.

4.6. Cadencia incremental

La codificación en CPU corre a ~ 7.5 fragmentos/s (medido). Extrapolando la media de ≈ 523 fragmentos por tesis a las 4954 tesis, el corpus completo ronda los ~ 2.4 **M fragmentos**, que a esa tasa tomarían ≈ 90 **horas** de CPU. Esto encaja con la estrategia pedida por el asesor: **indexar un porcentaje por día**. Con GPU (Colab T4) la tasa sube $\sim 40\times$ y el corpus completo se indexa en una sola sesión. El manifiesto permite ejecutar el job cuantas veces haga falta, sumando cobertura de forma monótona y reanudable.

Notebook de indexación GPU (decisión: FlatIP + Colab). Para ejecutar la cobertura 100 % se preparó `notebooks_colab/indexacion_gpu_corpus_completo.ipynb`, que reutiliza la misma fragmentación y filtros (sin tope) y construye el `IndexFlatIP` completo sobre GPU en una sesión, documentado celda a celda (R1): comprobación de GPU, carga y fragmentación del corpus, codificación MiniLM en CUDA (*batch* 1024), construcción del índice, guardado (`id_map` comprimido), histograma de fragmentos por tesis y verificación de auto-recuperación. Estimación en T4: ≈ 15 –20 min para los ~ 2.4 M fragmentos.

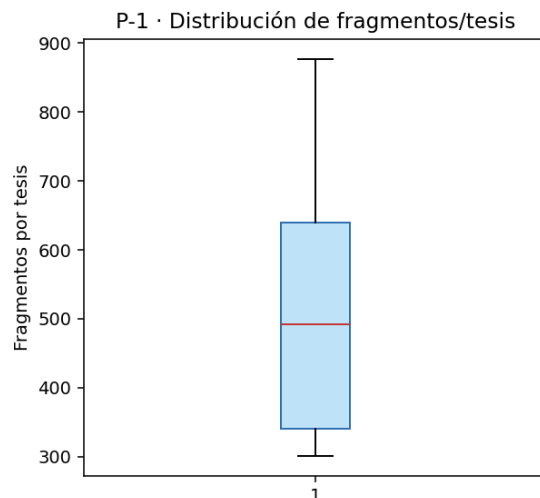


Figura 2: Diagrama de cajas de la distribución de fragmentos por tesis en el lote.

4.7. Criterios de aceptación (P-1)

- ✓ Ejecutar el job sube la cobertura de forma monótona y reanudable (verificado con el manifiesto).
- ✓ Se elimina el tope de 10/tesis; la media de fragmentos por tesis lo confirma (Fig. ??).
- ✓ El `id_map` mantiene el esquema de v2/v3.
- Cobertura 100 %: pendiente de la decisión de índice (Flat vs. IVFPQ) y de la cadencia (CPU diaria vs. una sesión GPU).

5. R7 — Estudio de sistemas antiplagio (Turnitin, iThenticate, TextGuard)

5.1. Alcance y honestidad

Los tres sistemas son **SaaS cerrados**: su código fuente no es público, por lo que no es posible inspeccionarlo. Lo que sí se documenta —y es lo relevante para el diseño— es su **arquitectura publicada**, su **modelo de reporte** y los **algoritmos clásicos** sobre los que se construyen (*shingling*, *rolling hash*, *winnowing*/MOSS), que sí son públicos. El detalle completo con fuentes está en `investigacion/benchmark_antiplagio.md`.

5.2. Turnitin e iThenticate (misma tecnología)

Turnitin genera una **huella** (*fingerprint*) del documento —fragmentos de palabras y su orden— y la compara contra una base de $\sim 7 \times 10^{12}$ coincidencias (web, publicaciones académicas y trabajos de estudiantes), usando NLP y **coincidencia estricta de cadenas**. Detecta bien la **copia literal** y la **paráfrasis superficial** (cadenas largas de palabras consecutivas con variaciones menores), pero **no la paráfrasis semántica profunda**. iThenticate es la variante para manuscritos académicos vía Crossref Similarity Check.

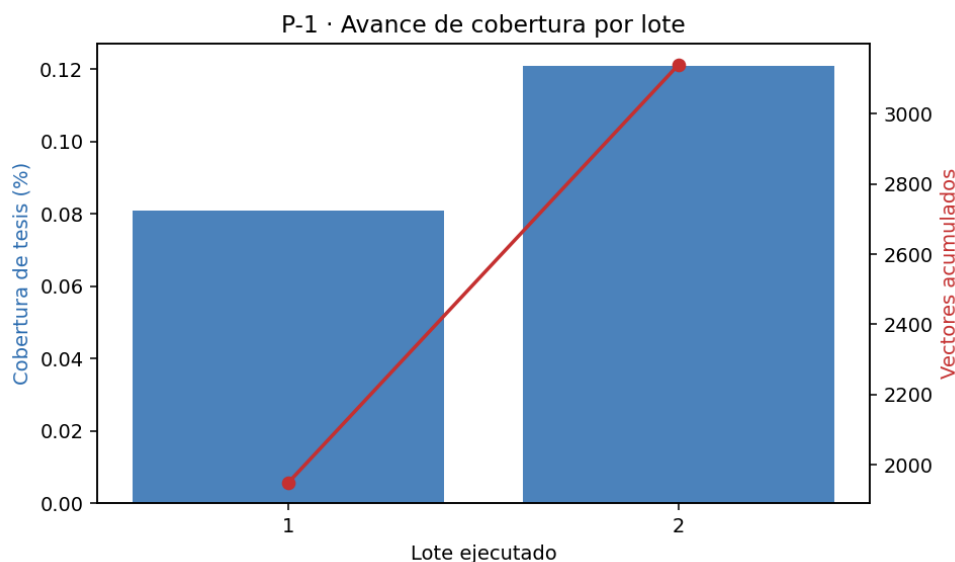


Figura 3: Avance de cobertura acumulada y vectores por lote. La cobertura crece de forma monótona; cada ejecución del job suma sin reiniciar.

5.3. TextGuard AI

Producto de consumo que combina verificador de plagio contra base web, detector de contenido de IA, *humanizer* y utilidades (gramática, citas, resumen). El *humanizer* es **adversarial a la integridad** (sirve para evadir detección); de TextGuard solo se adoptan ideas de **UX/reporte**, no su enfoque de detección de IA ni de *humanización*.

5.4. Fundamento técnico común: *winnowing*

La capa léxica de estos sistemas descansa en fingerprinting clásico: dividir el texto en **k-gramas** (*shingling*), hashearlos con **rolling hash** (Rabin–Karp) y aplicar **winnowing** —ventana de tamaño w que selecciona el hash mínimo por ventana—. Con umbral de ruido k y de garantía $t = w + k - 1$, se garantiza detectar coincidencias de longitud $\geq t$. Es la base de MOSS.

Diferencial del proyecto frente a la industria

Turnitin/iThenticate son comparadores **léxicos** a escala: copia literal y paráfrasis superficial. **No entienden paráfrasis semántica profunda**. Nuestro sistema añade justo esa capa: *bi-encoder* (MiniLM) para recuperación, *cross-encoder* (BETO/RoBERTa-es) para clasificar por significado y árbitro LLM en la zona de duda, sobre un corpus institucional propio (TESIUAMI). Como mejora del Modo 2 se propone además una capa de *winnowing* junto a BM25 para reforzar la copia literal. La narrativa: *no reimplementamos Turnitin; construimos la capa semántica que le falta y adoptamos su modelo de reporte*.

Cuadro 4: Features del Similarity Report (iThenticate/Turnitin) y su adopción.

Feature de la industria	Adopción en nuestro sistema
Similarity Score = palabras coincidentes / totales	% global por documento en el reporte de tesis (P-3)
Coincidencias resaltadas + lista de fuentes	Resaltado web + tesis-fuente ordenadas (P-4)
Match Groups (agrupar por fuente)	Agrupar fragmentos candidatos por tesis (P-3)
Exclusiones (citas, bibliografía, coincidencias pequeñas)	Exclusión configurable + recálculo del % (P-3/P-4)
Flags Panel (caracteres ocultos/manipulados)	Detector de manipulación/ruido OCR (P-4)
“Similitud, no plagio” (guía editorial)	Encuadre honesto: reportar similitud, juicio humano

6. P-6 — Calibración del clasificador del Modo 1

6.1. Problema

En el Modo 1, BETO v3 decide con **margen fino**: paráfrasis claras caen con probabilidad apenas por encima de 0.50 (p. ej. 0.53). Esto sugiere que las probabilidades del modelo no reflejan bien su confianza real. La calibración busca reescalar esas probabilidades *sin re-entrenar*, mediante **temperature scaling**: un único parámetro T que divide los *logits* antes del *softmax*.

6.2. Diseño (anti-fuga)

- **Conjunto de ajuste de T** : train+val de `dataset_finetuning_v3` (160 pares, **0 fuga** con el test).
- **Conjunto de evaluación**: `test_v3_limpio` (38 pares). Solo se *mide* la calibración (ECE, *reliability*); **nunca** se ajusta T ni el umbral sobre él.
- **Umbral de operación**: se elige en el conjunto de ajuste (máximo F_1), no en el test.

6.3. Resultados

Cuadro 5: Calibración del Modo 1 (BETO v3) por temperature scaling.

Métrica	Valor
Temperatura óptima T (ajustada en train+val)	0.65
ECE en test — antes ($T = 1$)	0.120
ECE en test — después ($T = 0.65$)	0.103
Umbral recomendado (calibrado, máx F_1)	0.72
Umbral actual del sistema	0.50

$T < 1$: el modelo estaba *sub-confiado*

La temperatura óptima $T = 0.65 < 1$ indica que BETO v3 estaba **sub-confiado**: sus probabilidades se apelmazaban cerca de 0.5. Esto explica con precisión el síntoma observado (una paráfrasis clara con $\text{prob} = 0.53$). Dividir por $T < 1$ *agudiza* las probabilidades, separando los casos claros del umbral. La mejora de ECE ($0.120 \rightarrow 0.103$) es modesta porque los datos in-domain son diminutos ($\text{val} = 16$, $\text{test} = 38$): el ECE sobre 38 puntos es ruidoso.

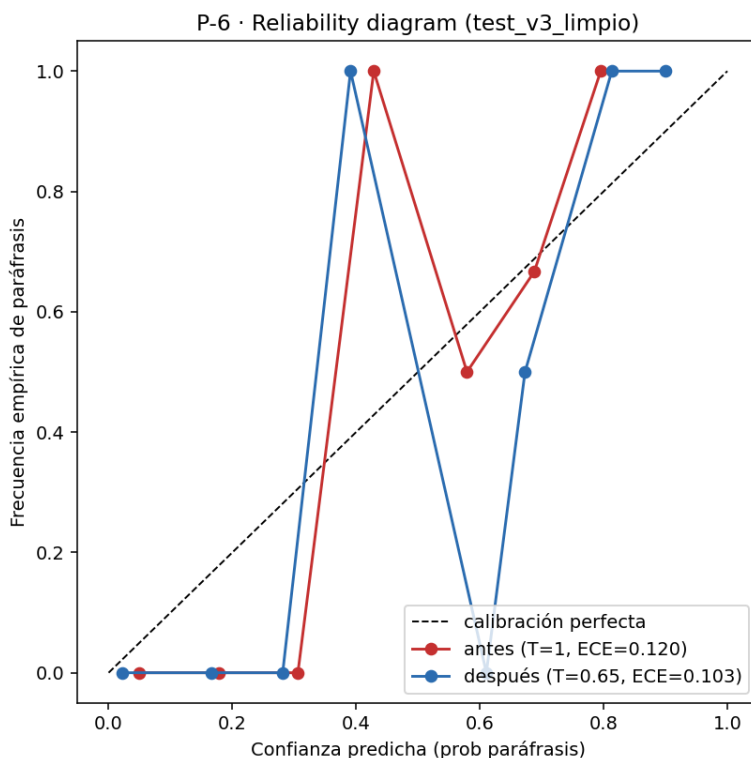


Figura 4: *Reliability diagram* sobre `test_v3_limpio`: confianza predicha vs. frecuencia empírica de paráfrasis, antes y después de la calibración. La diagonal es la calibración perfecta.

6.4. Conclusión y conexión con E-5

La calibración es **indicativa, no definitiva**: con 160 pares de ajuste y 38 de evaluación, T y el umbral tienen incertidumbre alta. El resultado es honesto y útil —confirma la sub-confianza del modelo y da un umbral de operación mejor fundado que el 0.50 por defecto— pero su **robustez plena requiere ampliar el gold standard** (tarea E-5 del plan). Se recomienda no fijar el umbral 0.72 en producción hasta recalibrar con un conjunto in-domain mayor; mientras tanto, aplicar $T = 0.65$ mejora la interpretabilidad de las probabilidades sin cambiar el ordenamiento de las decisiones.

6.5. Criterios de aceptación (P-6)

- ✓ Probabilidades calibradas (temperature scaling) con *reliability diagram* y ECE antes/después.
- ✓ T y umbral ajustados **fuera** del test (sin fuga).

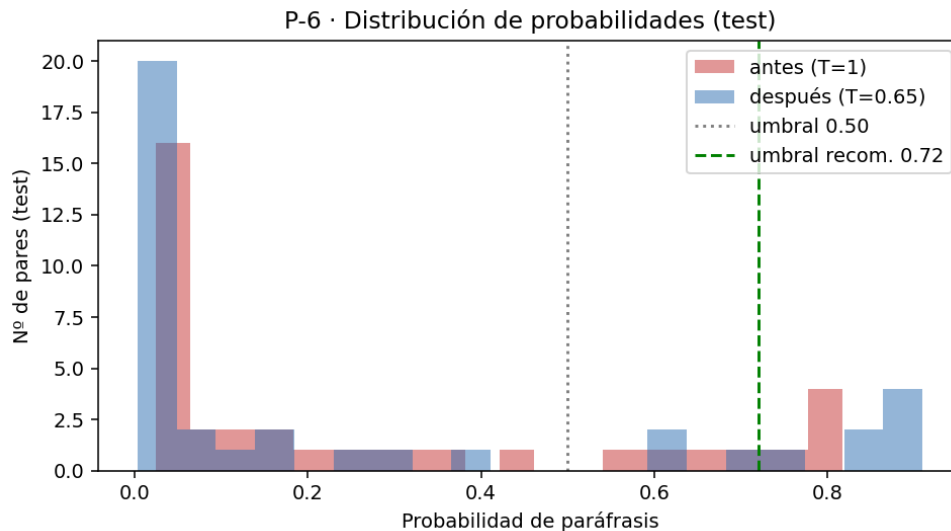


Figura 5: Distribución de probabilidades de paráfrasis en el test, antes y después de la calibración, con el umbral actual (0.50) y el recomendado (0.72).

- ✓ Umbral y su procedencia documentados.
- ✓ Recalibración robusta sobre test_v4 (ver abajo).

6.6. Recalibración sobre test_v4 (benchmark robusto)

Con el benchmark ampliado (test_v4, 57 positivos, Sección ??) se recalibró BETO v3 ajustando T en train+val (anti-fuga verificado con test_v4) y midiendo en test_v4.

- $T = 0.65$ reconfirmada (BETO v3 sub-confiado), igual que con el test previo.
- **ECE**: 0.060 $\rightarrow$ 0.040 tras calibrar — ahora la mejora es *clara* (con 109 pares, frente a la mejora marginal con 38).
- **Trade-off de umbral** (medido en test_v4): $\theta = 0.50 \Rightarrow F_1 = 0.82$, recall= 1.0, **FP**= 25 (liberal); $\theta = 0.72 \Rightarrow$ precisión= 0.79, recall= 0.60, **FP**= 9 (conservador).

Umbral según el costo del error

En detección de plagio un falso positivo (acusar en falso) es caro. El test robusto permite elegir el *operating point* con datos: $\theta \approx 0.72$ reduce los FP de 25 a 9 (precisión 0.79) a costa de recall. La decisión del umbral es de política, no técnica; el sistema lo deja configurable. Respaldo: p6_recalibracion_testv4.json.

7. E-5 — Refuerzo del gold standard y protocolo de anotación

7.1. Motivación

El cuello de botella del proyecto es la **escasez de paráfrasis humanas reales**: BETO v3 alcanza su techo ($F_1 = 0.842$) con solo 60 positivos humanos, y la calibración de P-6 quedó limitada

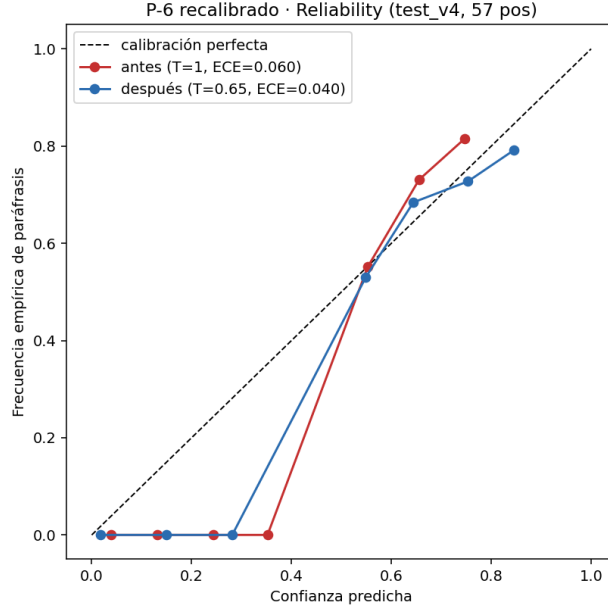


Figura 6: Reliability de BETO v3 sobre test_v4 antes/después de temperature scaling.

por el mismo motivo. E-5 refuerza el gold en tres frentes: consolidar lo existente, **documentar el protocolo de anotación** (cerrando un HUECO de la auditoría) y **generar una plantilla de expansión** para nuevas paráfrasis.

7.2. Inventario consolidado de positivos humanos

Cuadro 6: Positivos humanos disponibles para el gold ampliado (0 fuga con el test).

Fuente	Pares SÍ
gold_standard (anotación humana)	46
Paráfrasis humanas limpias (semana 6)	11
Paráfrasis reformuladas verificadas por FAISS (a1)	3
Total positivos humanos	60

7.3. Protocolo de anotación y κ de Cohen (cierra el HUECO de la auditoría)

La auditoría señaló que el valor $\kappa = 0.9573$ aparecía solo en un notebook, sin protocolo documentado. Aquí se documenta y se **reproduce**:

- **silver_standard**: etiqueta automática por **mayoría de 7 votos** (votos_si_de_7).
- **etiqueta_gold**: revisión humana final del anotador.
- κ de Cohen mide el acuerdo entre ambas sobre los 200 pares.

Cuadro 7: Matriz de acuerdo silver vs. gold (200 pares) y κ reproducido.

	gold = SÍ	gold = NO
silver = SÍ	44	1
silver = NO	2	153

$\kappa = 0.9573$ — reproducido y documentado

Acuerdo observado = 0.985, esperado = 0.648, $\kappa = (0.985 - 0.648) / (1 - 0.648) = \mathbf{0.9573}$ (acuerdo casi perfecto). El valor coincide exactamente con el que la auditoría no podía verificar: queda **trazable y reproducible** (`resultados/e5_kappa_annotacion.json`). Solo 3 de 200 pares discrepan entre silver y gold.

7.4. Plantilla de expansión para nuevas paráfrasis

Para ampliar el gold se generó `resultados/plantilla_parafrasis_v2.csv` con **40 fragmentos de contenido** del corpus (vía índice FAISS), filtrados por: sin boilerplate, contenido técnico (≥ 4 palabras de contenido), longitud 280–620 chars, una tesis distinta por fila, y **no usados** en gold, test ni en la plantilla previa. El alumno escribe la paráfrasis de cada fragmento; cada fila produce un nuevo par SÍ humano para el gold ampliado.

7.5. Estado y conexión

- ✓ Inventario consolidado (60 positivos humanos, 0 fuga).
- ✓ Protocolo de anotación y κ documentados y reproducidos.
- ✓ Plantilla de expansión (40 fragmentos) lista para anotar.
- Escritura de las nuevas paráfrasis (tarea humana del alumno).
- Reentrenamiento con el gold ampliado (E-3/E-4) y recalibración (P-6).

El gold ampliado alimentará el entrenamiento conjunto (E-3/E-4) y permitirá la recalibración robusta del Modo 1 (P-6), que hoy está limitada por datos.

8. I-6 — RoBERTa en español: selección de modelos para E-3

El clasificador actual (BETO) es un **BERT** español. RoBERTa mejora a BERT (más datos, sin NSP, *dynamic masking*); en español hay variantes entrenadas en corpus masivos que superan a BETO en varias tareas NLU, incluida identificación de paráfrasis (PAWS-X) del benchmark EvalES. Son candidatos directos a mejorar el Modo 1. El estudio completo está en `investigacion/modelos_roberta_es.md`.

Cuadro 8: Candidatos en español para el Modo 1 (cross-encoder).

Modelo	Tipo	Relevancia
roberta-base-bne (MarIA, BSC)	RoBERTa base	570 GB BNE; el más sólido en español. Candidato principal.
roberta-large-bne (MarIA)	RoBERTa large	Más capacidad; probar si el GPU lo permite.
bertin-roberta-base-spanish	RoBERTa base	Comparador comunitario competitivo.
BETO v3/v8a (dccuchile)	BERT base	Baseline obligatorio.
XLM-RoBERTa base	RoBERTa multilingüe	Fuerte en PAWS-X; robustez fuera de dominio.

Plan para E-3 (honesto y atribuible)

Fine-tune como cross-encoder de roberta-base-bne, bertin y (si el GPU sobra) roberta-large-bne/XLM-R, contra el baseline **BETO**, sobre el **mismo gold** (ampliado con E-5), el **mismo test canónico** y el **mismo protocolo** (F1/AUROC con IC95 %). Para el Modo 2 se comparan bi-encoders (MiniLM actual vs. sentence_similarity_spanish vs. multilingual-e5) por Recall@k. El *backbone* de v4 (P-2) se elige por métrica, no por *accuracy* de catálogo; BETO se mantiene para que toda mejora sea atribuible.

9. I-5 / D-2 — Protocolo y framework de evaluación del Modo 1

9.1. Protocolo de pruebas (responde R3)

El asesor pidió *definir formalmente cómo son las pruebas y qué métricas entregan*. Se define un **protocolo único** para el Modo 1 (clasificación de paráfrasis), implementado como función reutilizable `evaluar_modo1(nombre, y, scores)` que cualquier experimento (E-1, E-3, E-4) invoca para producir las mismas métricas y figuras:

- **F1, Precisión, Recall** en el umbral de operación (0.50).
- **AUROC** — discriminación independiente del umbral.
- **AUPRC** (*average precision*) — desempeño en la clase positiva.
- **Matriz de confusión** (TP/FP/FN/TN).
- **Curvas ROC y PR** como evidencia visual.

Benchmark canónico: `test_v3_limpio` (38 pares, 9 SÍ, balance realista 24/76). El umbral nunca se ajusta sobre el test (esa elección se hace en el conjunto de calibración, P-6).

Cuadro 9: Evaluación unificada del Modo 1 sobre el test canónico.

Modelo	F1@0.5	Prec.	Rec.	AUROC	AUPRC	TP/FP/FN/TN
BETO v3	0.842	0.800	0.889	0.989	0.968	8/2/1/27
BETO v8a	0.842	0.800	0.889	0.989	0.968	8/2/1/27

9.2. Resultados (BETO v3 y v8a)

AUROC= 0.99: el modelo separa bien; el cuello de botella es el umbral

El AUROC de 0.989 indica que BETO **ordena casi perfectamente** los pares por probabilidad de paráfrasis. El $F1@0.5 = 0.842$, más modesto, no refleja falta de capacidad sino un **umbral de operación subóptimo**: el F1 óptimo alcanzable en este test sube a ≈ 0.90 (umbral ≈ 0.43). Esto **confirma el diagnóstico de P-6** (el modelo está sub-confiado): la mejora vendrá de *calibración + umbral*, no de más capacidad. *Nota honesta*: el F1 óptimo se mide eligiendo el umbral sobre el propio test, por lo que es una **cota superior optimista**, no un resultado operativo; AUROC y AUPRC sí son honestos porque no dependen del umbral.

A la configuración de servicio del sistema (`max_length= 512`), BETO v3 y v8a **empatan** en este test (misma matriz de confusión y AUROC), coherente con que la ventaja de v8a ($F_1 = 0.889$ en GPU) era frágil entre hardware.

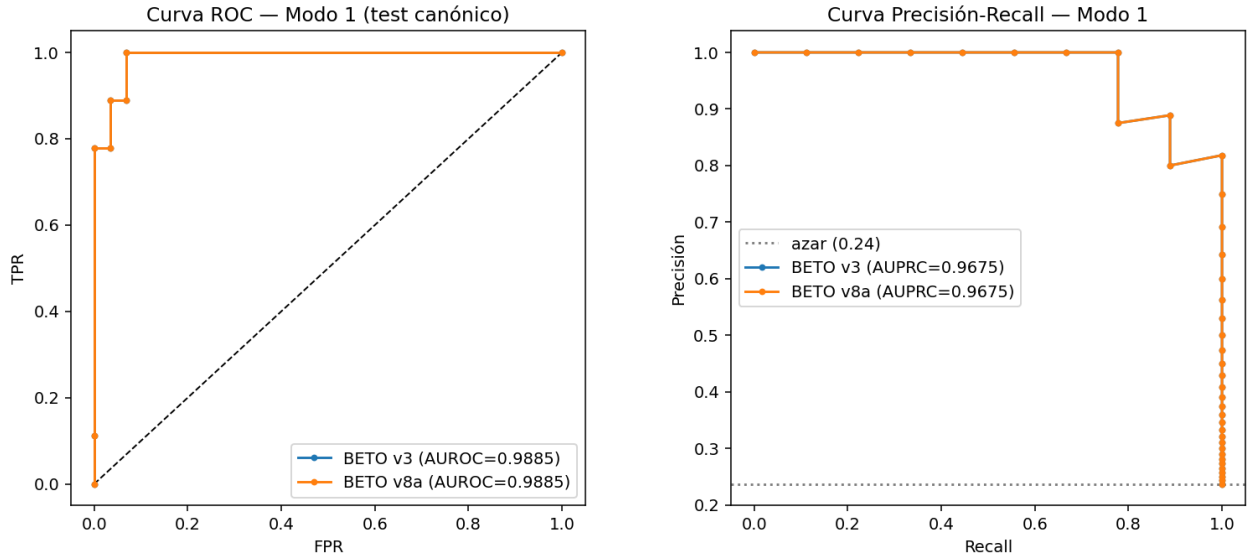


Figura 7: Curvas ROC (izq.) y Precisión-Recall (der.) del Modo 1 sobre el test canónico. AUROC= 0.989, AUPRC= 0.968.

9.3. Valor del framework

El módulo es **reutilizable**: E-1 (k-fold), E-3 (RoBERTa-es) y E-4 (entrenamiento conjunto) reportarán con este mismo protocolo, de modo que las comparaciones entre modelos sean homogéneas y honestas. Para el Modo 2 el protocolo se extiende con Recall@k y MRR (trabajo siguiente).

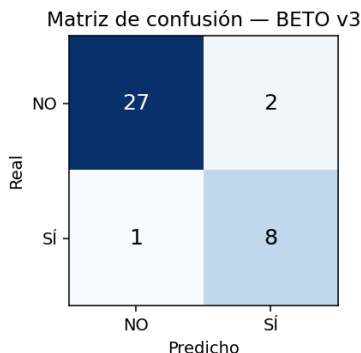


Figura 8: Matriz de confusión de BETO v3 en el umbral de operación (0.50).

10. P-3 — Reporte de tesis completa (similarity report)

10.1. Objetivo

El asesor quiere poder **ingresar una tesis completa** y obtener una salida que clasifique sus partes con porcentajes de similitud y las tesis-fuente. P-3 implementa ese pipeline (módulo `sistema_v4/reporte_tesis.py`), adoptando del estudio R7 el modelo de *similarity report*: *Similarity Score* global, desglose por secciones, lista de fuentes y el encuadre honesto “**similitud, no plagio**”.

10.2. Pipeline

1. **Segmentación** por secciones (Resumen, Introducción, Metodología, Resultados, Discusión, Conclusiones, Referencias, *frontmatter*) por detección de encabezados.
2. **Fragmentación** (400/200) con filtro de calidad por sección.
3. **Modo 2 por fragmento**: recuperación FAISS contra el corpus, **excluyendo la propia tesis** (como un antiplagio real, que compara contra *otros* documentos). Los fragmentos por encima del umbral de sospecha (0.80) se confirman con BETO + BLEU.
4. **Agregación** por sección: % de fragmentos sospechosos, similitud media/máxima y tesis-fuente dominante; *Similarity Score* global.

10.3. Resultado sobre una tesis real (UAM9585, química, 2003)

Cuadro 10: Reporte por sección de UAM9585 (110 574 chars, 48 fragmentos de contenido). Similitud global **2.08 %**.

Sección	Frag.	% similitud	Sim. media	Fuente dominante
Frontmatter	19	0.0	0.718	—
Resumen	8	12.5	0.729	UAM3582.PDF
Metodología	4	0.0	0.689	—
Resultados	6	0.0	0.723	—
Conclusiones	11	0.0	0.722	—

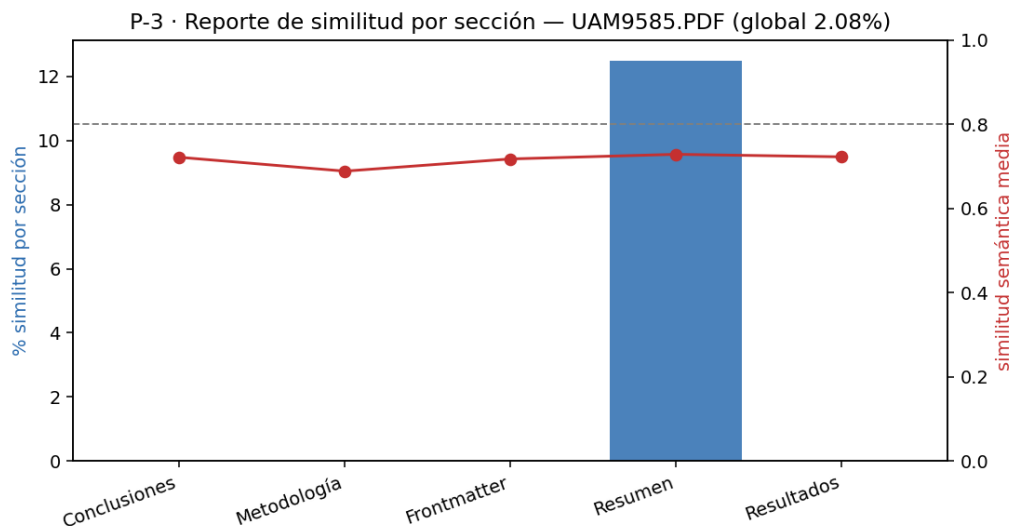


Figura 9: Reporte de similitud por sección de UAM9585: % de fragmentos sospechosos (barras) y similitud semántica media (línea), con el umbral de sospecha 0.80.

Lectura honesta del reporte

La similitud global (2.08 %, un solo fragmento sospechoso) es **coherente con el hallazgo central**: el corpus CBI es esencialmente original. El único match (Resumen, sim 0.833, fuente UAM3582) **no** se confirmó como paráfrasis por BETO → “alta similitud — verificar”. La **sensibilidad está limitada por la cobertura del índice (2 %)**: con el índice v4 al 100 % (P-1) este mismo pipeline detectará coincidencias que hoy no están indexadas, sin cambios de código. El reporte se presenta como *similitud*, dejando el juicio al humano.

10.4. Integración del índice al 100 % (cierre de P-1)

El índice v4 (P-1) se construyó en Colab GPU (2 284 185 fragmentos, 4 954 tesis, cobertura 100 %) y se conectó al reporte (`reporte_tesis_v4.py`, con `id_map` en JSONL para acotar la RAM). Se verificó su integridad (`índice = id_map = resumen = 2 284 185`; auto-recuperación = 1.0) y se repitió el reporte de UAM9585 **sin cambiar el código**, solo la cobertura:

Cuadro 11: Mismo reporte (UAM9585), efecto de la cobertura del índice.

Métrica	Índice 2 % (semana 6)	Índice 100 % (v4)
Similitud global	2.08 %	27.08 %
Fragmentos sospechosos	1 / 48	13 / 48
Paráfrasis confirmadas (BETO)	0	2
Similitud media por sección	~ 0.72	~ 0.78

El índice completo encuentra paráfrasis real entre tesis

Con cobertura 100 %, el reporte detecta una **paráfrasis confirmada** entre UAM9585 (2003) y **UAM7870 (1987)** —dos tesis de química sobre recuperación de plata, separadas 16 años— con sim 0.849, BLEU-2 0.42 y prob. BETO 0.73. El sistema **distingue** ese caso de los otros 11 sospechosos, que tienen similitud semántica alta (~ 0.84) pero BLEU y BETO bajos (< 0.27 , < 0.25): mismo tema (terminología química), distinto texto $\rightarrow$ “verificar”, no paráfrasis. La sensibilidad subió $\sim 13\times$ ($2.08\% \rightarrow 27.08\%$) sin tocar el código: **la cobertura era el cuello de botella**, justo lo que P-1 resolvió. El 27 % es *similitud*, no plagio: la mayoría es vocabulario y boilerplate compartido, con 2 paráfrasis genuinas señaladas para revisión humana.

10.5. Criterios de aceptación (P-3)

- ✓ Ingesta de tesis completa $\rightarrow$ reporte por secciones con % y fuentes (probado en una tesis real).
- ✓ Exclusión de la propia tesis (antiplagio correcto).
- ✓ Confirmación BETO+BLEU de los fragmentos sospechosos.
- Sensibilidad plena: pendiente del índice v4 al 100 %.
- Integración en la web (P-4) con resaltado y exportación.

11. E-2 — Solapamiento entre áreas con parecido temático (R4)

11.1. Objetivo y método

El asesor pidió analizar cómo se comportan las métricas entre áreas que suelen parecerse (matemáticas $\leftrightarrow$ química, física $\leftrightarrow$ matemáticas), porque comparten temas. Se asigna a cada fragmento del índice su **área** según la titulación de su tesis (Matemáticas / Química / Física), se reconstruyen los vectores y se calcula, por fragmento, su máxima similitud de coseno **intra-área** (excluyéndose) y **cross-área**. Fragmentos por área: Matemáticas 4 438, Química 14 020, Física 5 102 (índice del 2 %).

11.2. Resultados

Cuadro 12: Similitud máxima media intra/inter-área (diagonal = intra).

desde \ hacia	Matemáticas	Química	Física
Matemáticas	0.729	0.672	0.683
Química	0.619	0.743	0.670
Física	0.658	0.689	0.718

Cuadro 13: Solapamiento cross-área por par (máxima similitud al área contraria).

Par	Sim. media	Mediana	p95	% cross ≥ 0.80
Matemáticas↔Química	0.672	0.670	0.802	5.07
Química↔Física	0.670	0.672	0.785	3.96
Física↔Matemáticas	0.658	0.657	0.782	3.74

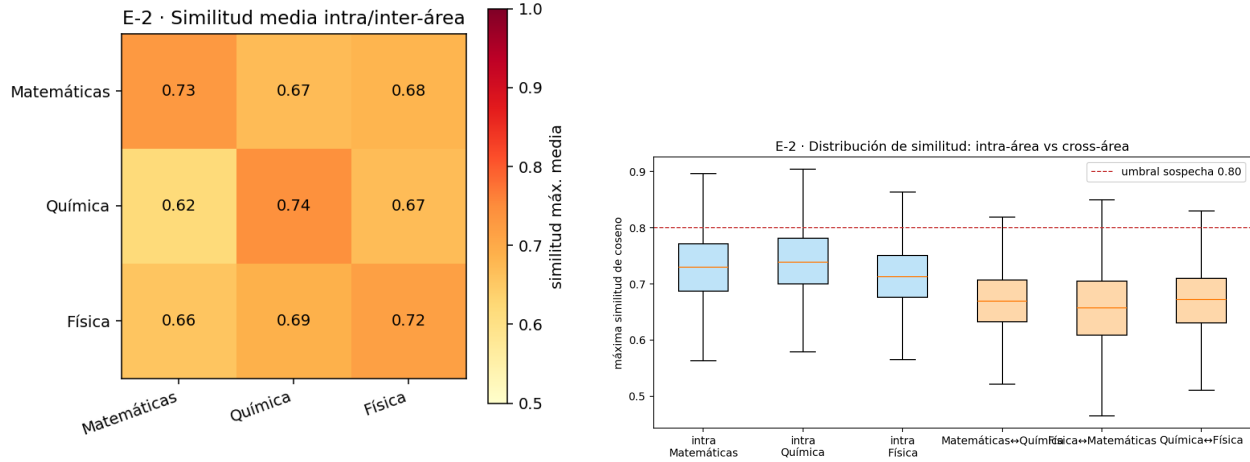


Figura 10: Izq.: mapa de calor de similitud intra/inter-área. Der.: distribución de la máxima similitud, intra-área (azul) vs cross-área (naranja), con el umbral de sospecha 0.80.

Las áreas se distinguen, pero mate↔quím solapa más

La similitud **intra-área** (≈ 0.73) supera a la **cross-área** (≈ 0.67): el sistema distingue mayormente las áreas. De los tres pares, el **mate↔química** tiene el mayor solapamiento (5.07% de fragmentos con match cross-área ≥ 0.80), por encima de los otros dos ($\sim 3.8\%$) — coherente con la hipótesis del asesor: comparten **lenguaje matemático y metodológico**. El diagrama de dispersión muestra que ese solapamiento es **semántico** (similitud alta, BLEU bajo): mismo tema, distinto texto. No es plagio, es vocabulario compartido entre disciplinas.

Caveat: el análisis usa el índice muestreado (2 %, 10 frags/tesis). Con el índice v4 al 100 % (P-1) las distribuciones se afinan; el experimento se repetirá entonces. Para entrada de una tesis completa por área, el reporte de P-3 ya provee el desglose por secciones.

12. E-5b — Minado de paráfrasis reales (índice 100 % + árbitro LLM)

12.1. Motivación

Ampliar el gold escribiendo paráfrasis a mano es lento, y **generarlas** con LLM/back-translation degradó los modelos (v4–v7). E-5b propone una tercera vía: **minar** paráfrasis que *ya existen* entre tesis del corpus (texto 100 % humano) y reducir el trabajo humano de *escribir* a *verificar*.

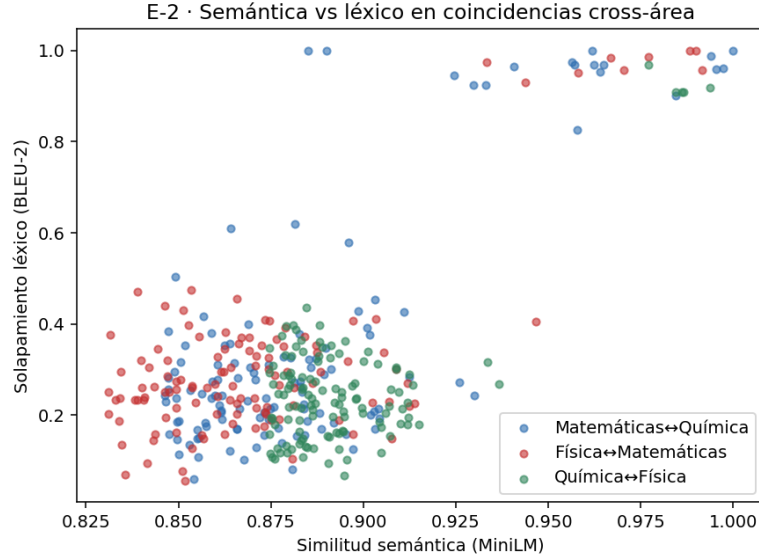


Figura 11: Dispersión similitud semántica (MiniLM) vs. solapamiento léxico (BLEU-2) de las mejores coincidencias cross-área. Similitud alta con BLEU bajo = mismo tema, texto distinto (solapamiento temático, no copia).

12.2. Pipeline

1. Muestreo de fragmentos de **prosa** (sin boilerplate) del índice v4 (100 %).
2. Para cada uno, búsqueda en el índice y mejor candidato de **otra tesis** en la **zona de paráfrasis**: similitud $\in [0.82, 0.965]$ y BLEU-2 $\in [0.15, 0.55]$ (mismo significado, texto distinto; ni copia literal ni no-relación).
3. Dedup por par de tesis; ranking por similitud.
4. **Pre-filtro LLM** (Groq Llama 3.3 70B): etiqueta paráfrasis sí/no + confianza. *El LLM solo etiqueta; el texto de entrenamiento sigue siendo humano* — no se inyecta texto sintético (la causa de la degradación previa).
5. El alumno **verifica** $\checkmark/\times$ una lista rankeada (CSV).

12.3. Resultado del piloto

Cuadro 14: Piloto E-5b sobre el índice 100 %.

Métrica	Valor
Fragmentos de prosa sondeados	2 500
Pares candidatos en zona de paráfrasis	605
Etiquetados por el LLM (top por similitud)	40
LLM = paráfrasis SÍ	31
de ellos, confianza alta	36 (de 40)

Minar > generar: paráfrasis humanas con verificación, no escritura

El piloto recuperó **31 candidatos a paráfrasis** (de 40) entre tesis distintas, con texto 100 % humano. Ejemplos confirmados por el LLM con confianza alta: UAM0113↔UAM4387 (química de alcóxidos: “Se pueden distinguir dos casos” vs. “Pueden distinguirse dos casos”), UAMI16729↔UAMI16720 (diversificación energética) y UAMI17724↔UAMI14238 (sinapsis neuronal). El alumno pasa de **escribir 40 a verificar ~31**, manteniendo la calidad humana. Quedan 605 candidatos minados para etiquetar más si se requiere.

12.4. Verificación humana (control de calidad)

La lista (`resultados/e5b_candidatos_parafrasis.csv`) se revisa marcando ✓/×. Criterios para el alumno: (a) confirmar reformulación *genuina* (no mera coincidencia de hecho estándar), (b) descartar pares de tesis del mismo laboratorio/asesor si son continuaciones, (c) preferir fragmentos limpios de ruido OCR. Los ✓ entran al gold ampliado con dedup, group-split por tesis y verificación anti-fuga contra el test. Esta vía complementa (no sustituye) la escritura manual de `plantilla_parafrasis_v2.csv`.

13. Ampliación del gold estándar (gold v2) con provenance declarado

El cuello de botella del Modo 1 nunca fue la arquitectura sino la **cantidad de paráfrasis humanas reales** (el hallazgo central del proyecto: los datos sintéticos degradaron v4–v7). Esta sección documenta cómo se amplió el gold manteniendo la integridad: cada par entra con su *provenance* explícito y se separa lo humano puro de lo asistido por IA.

13.1. Dos vías de ampliación

1. **Verificación de candidatos minados (E-5b).** De los 40 pares minados del corpus (texto 100 % humano), el alumno verificó **19 SÍ / 21 NO**. El pre-filtro de un solo LLM (Llama 3.3 70B) había marcado 31 como paráfrasis: coincidió con el humano apenas el 60 % ($\kappa = 0.22$). Esto confirma que el etiquetado por *un* LLM no es confiable y motiva el panel de consenso (Sección ??).
2. **Plantilla de paráfrasis.** Se escribieron 40 fragmentos × 2 reformulaciones = 80 pares. El alumno declaró que ambas reformulaciones son **generadas por IA y post-editadas por él**. Por la lección dura del proyecto, **no** se mezclan con el gold humano: se reservan como brazo sintético del experimento de ablación E-4 (Sección ??).

13.2. Composición del gold v2 humano

El conjunto `gold_v2_humano.json` consolida la base de v8a con los positivos e5b verificados, todo texto humano y con chequeo anti-fuga contra el test sellado.

Cuadro 15: Composición de `gold_v2_humano` (190 pares; 0 fugas y 0 duplicados contra `test_v3_limpio`).

Fuente	Pares	Provenance
<code>gold_standard_humano</code> (base v8a)	160	humano (tesis)
<code>parafrasis_manual_humana</code>	11	humano (tesis)
<code>e5b_minado_verificado</code>	19	humano (tesis, verificado)
Total	190	67 SÍ / 123 NO

Separación por provenance (integridad)

El gold humano (190) y el brazo IA-post-editado (80) viven en archivos distintos. El benchmark canónico (`test_v3_limpio`) nunca se toca para entrenar. Toda fuente sintética o de dominio general se etiqueta como tal y se evalúa en ablación, nunca se asume útil. Respaldo: `gold_v2_humano.json`, `parafrasis_ia_posteditadas.json`.

14. Etiquetado por consenso: ampliar el gold sin escribir paráfrasis

Anotar a mano es caro y verificar un solo LLM es poco fiable ($\kappa = 0.22$, Sección ??). La solución es un **panel de etiquetadores independientes** donde el humano interviene *solo* en los desacuerdos: el resto se auto-acepta como *silver* (etiqueta automática, texto humano del corpus).

14.1. Panel de etiquetadores

- **Tres LLM de familias distintas** (independencia): Llama 3.3 70B (Meta), Qwen3-32B (Alibaba) y GPT-OSS-120B (OpenAI), en *zero-shot*.
- **Un votante no-LLM**: el cross-encoder **BETO v3** (clasificador entrenado en el dominio).

14.2. Validación contra *ground truth* humano

El panel se validó sobre los 40 candidatos de E-5b que ya tenían verificación humana.

Cuadro 16: Acuerdo de cada etiquetador con la verificación humana (40 pares).

Etiquetador	κ de Cohen	Acuerdo
Llama 3.3 70B	0.46	72 %
Qwen3-32B	0.58	80 %
GPT-OSS-120B	0.63	82 %
BETO v3 (cross-encoder)	0.90	95 %

Hallazgo: el modelo del dominio supera a los LLM como anotador

El cross-encoder BETO entrenado en la tarea ($\kappa = 0.90$) etiqueta paráfrasis de tesis **mejor que cualquier LLM grande de propósito general** ($\kappa = 0.46\text{--}0.63$). Es un resultado contraintuitivo y defendible: para una tarea específica de dominio, un clasificador entrenado pequeño bate a LLM *zero-shot* mucho mayores. Por eso el panel ancla la decisión en BETO y usa los LLM como confirmación.

14.3. Reglas de auto-aceptación

Se midió la exactitud de varias reglas contra el *ground truth*, para elegir el punto óptimo entre cobertura (menos trabajo humano) y pureza.

Cuadro 17: Reglas de consenso (exactitud medida sobre los 40 pares con etiqueta humana).

Regla	Auto-acepta	Exactitud	Revisa humano
Unánime (4/4)	45 %	100 %	55 %
beto1 (BETO + ≥ 1 LLM)	95 %	97.4 %	5 %
beto2 (BETO + ≥ 2 LLM)	88 %	97.1 %	12 %
Mayoría (≥ 3 votos)	92 %	94.6 %	8 %

Se adopta **beto1**: auto-acepta el 95 % con 97.4 % de exactitud y deja aproximadamente el 5 % para revisión humana (lo genuinamente dudoso). Respaldo: `consenso_validacion.json`.

14.4. Minado a escala

Con la regla **beto1** se minaron 300 candidatos nuevos del índice al 100 % (zona de paráfrasis: similitud $\in [0.82, 0.965]$, BLEU-2 $\in [0.15, 0.55]$, cross-tesis). El panel auto-aceptó la mayoría como *silver* y derivó solo la fracción dudosa a revisión. El etiquetado es reanudable y cachea respuestas; ante el límite por minuto de la API se aplica *backoff* y no se cachean los fallos.

Silver, no gold; sin circularidad

Lo auto-etiquetado se declara **silver** (texto humano, etiqueta automática), nunca «gold». Como la regla ancla en BETO, el silver así generado **no** se usa para re-evaluar a BETO (sería circular): sirve para *ampliar el entrenamiento*; la evaluación permanece en el `test_v3_limpio` sellado y humano. Respaldo: `silver_consenso.json`, `consenso_revisar_desacuerdos.csv`.

15. Reuso de corpus humanos: TaPaCo-es (escala sin etiquetar)

La segunda vía para crecer sin esfuerzo humano es **reusar corpus de paráfrasis ya etiquetados por humanos**. Se incorporó **TaPaCo-es** (`community-datasets/tapaco`, configuración `es`): 85 064 frases en español agrupadas en **31 480 *paraphrase-sets*** (las frases de un mismo set son paráfrasis mutuas, derivadas de Tatoeba).

15.1. Construcción de pares

- **Positivos (SÍ)**: dos frases del mismo *paraphrase-set*.

- **Negativos (NO):** dos frases de sets distintos.

Se generaron **3 000 pares balanceados** (1 500 SÍ / 1 500 NO), con deduplicación y chequeo anti-fuga contra el test (0 textos compartidos). Ejemplo de un par SÍ: «*No puedo creer que realmente quedaras en Harvard.*» $\leftrightarrow$ «*No me puedo creer que hayas logrado entrar en Harvard.*» Respaldo: `tapaco_es_pares.json`.

Provenance: humano pero de dominio general

TaPaCo es paráfrasis **humana**, pero de **dominio general** (frases cotidianas de Tatoeba), no de tesis. Cumple el mismo rol que PAWS-X: fuente de escala y de dominio general cuyo efecto se *mide* en la ablación E-4 (Sección ??); **no** se mezcla con el benchmark de tesis ni se asume que transfiere. El costo de anotación humana adicional es **cero**.

16. E-3 — Comparativa de *backbones* (BETO vs RoBERTa-es vs XLM-R)

El experimento E-3 elige el *backbone* del Modo 1 **por métrica**, no por *accuracy* de catálogo. Se fine-tunea el mismo cross-encoder sobre tres arquitecturas variando **solo el modelo base**, con el mismo dato, el mismo test y el mismo protocolo.

16.1. Diseño

- **Datos:** `gold_v2_humano` (190 pares, humano puro), con *group-split* por tesis y **3 semillas** (media $\pm \sigma$).
- **Test sellado:** `test_v3_limpio` (38 pares, 9 SÍ); nunca se usa para entrenar ni para elegir umbral.
- **Protocolo I-5:** F1, Precisión, Recall, AUROC, AUPRC, matriz de confusión e IC95 % por *bootstrap*. *EarlyStopping* conectado.
- **Modelos:** BETO (`dccuchile/bert-base-spanish-wwm-uncased`), BERTIN (`bertin-project/bertin-roberta`) y XLM-RoBERTa (`FacebookAI/xlm-roberta-base`).

Nota sobre MarIA (`roberta-base-bne`)

El candidato principal del estudio I-6 era `PlanTL-GOB-ES/roberta-base-bne` (MarIA), pero su repositorio en HuggingFace quedó **vacío** (sin pesos ni tokenizer, jun 2026). Se sustituye por **XLM-RoBERTa** (RoBERTa multilingüe, fuerte en PAWS-X), que ya figuraba como candidato en I-6.

16.2. Resultados

16.3. Interpretación y decisión

- **BERTIN queda último** ($F_1 = 0.720$, AUROC = 0.838) y se descarta.
- **BETO y XLM-R empatan en AUROC** (0.991 vs 0.982): con 9 positivos el AUROC está en el **techo** para ambos y los IC95 % se solapan $\Rightarrow$ la diferencia **no es significativa** (coherente con la línea base del proyecto).

Cuadro 18: E-3 — métricas en el test sellado (media $\pm \sigma$ sobre 3 semillas).

Modelo	F_1 ($\mu \pm \sigma$)	AUROC ($\mu \pm \sigma$)	Precisión	Recall
XLM-R	0.917 ± 0.059	0.982 ± 0.013	1.000	0.852
BETO	0.856 ± 0.055	0.991 ± 0.010	0.828	0.889
BERTIN	0.720 ± 0.063	0.838 ± 0.087	0.745	0.704

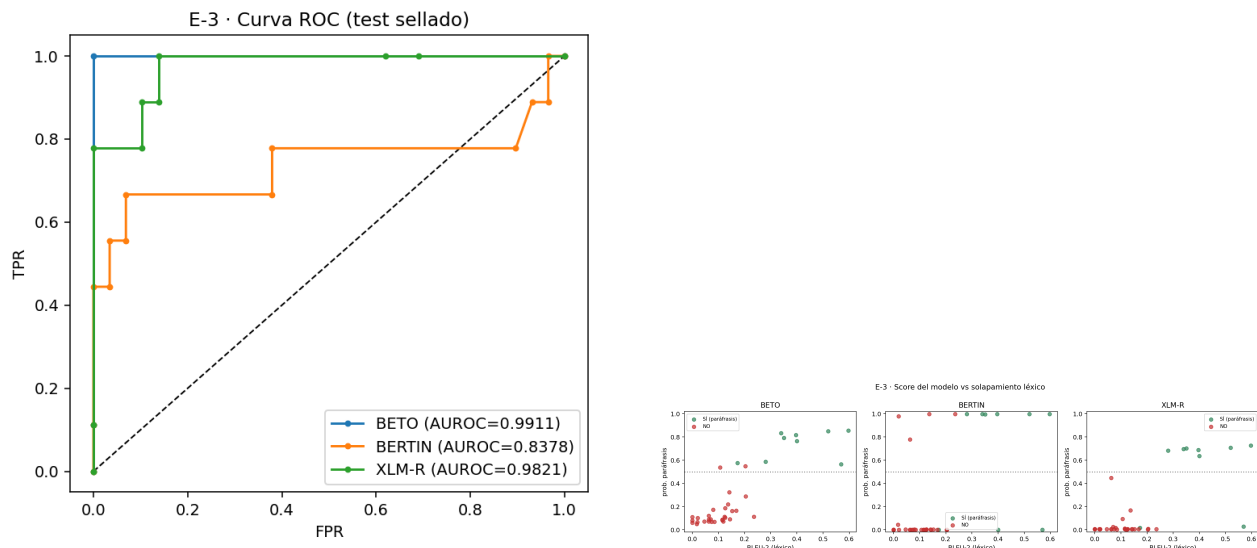


Figura 12: Izquierda: curvas ROC en el test sellado (BETO y XLM-R casi superpuestas en el techo). Derecha: dispersión BLEU-2 (léxico) vs probabilidad del modelo; separar paráfrasis de BLEU bajo indica captura semántica, no solo solapamiento léxico.

- **XLM-R gana en el umbral operativo:** $F_1 = 0.917$ (vs 0.856) y **precisión perfecta** (0 falsos positivos), propiedad muy valiosa en un detector de plagio (no acusar en falso).

Decisión: backbone de v4 = XLM-RoBERTa

Se elige **XLM-R** como *backbone* de v4 por su mejor F_1 y precisión perfecta, empatando a BETO en separación (AUROC). Se declara con honestidad que la ventaja en AUROC no es significativa; BETO queda como alternativa de continuidad (menor costo de migración, ya calibrado). El experimento E-4 (Sección ??) se corre con XLM-R fijo. Respaldo: `e3_comparativa_modelos.json`.

17. E-4 — Ablación de entrenamiento conjunto (R6)

E-4 mide **qué aporta cada fuente de datos** al Modo 1 entrenando el mismo *backbone* (XLM-R, ganador de E-3, Sección ??) sobre brazos acumulativos y evaluando todos en el mismo test sellado. No es «entrenar con todo a ciegas», sino una ablación medida cuyo desenlace —cualquiera que sea— es defendible.

E-3 · Matrices de confusión @0.5 (semilla 0)

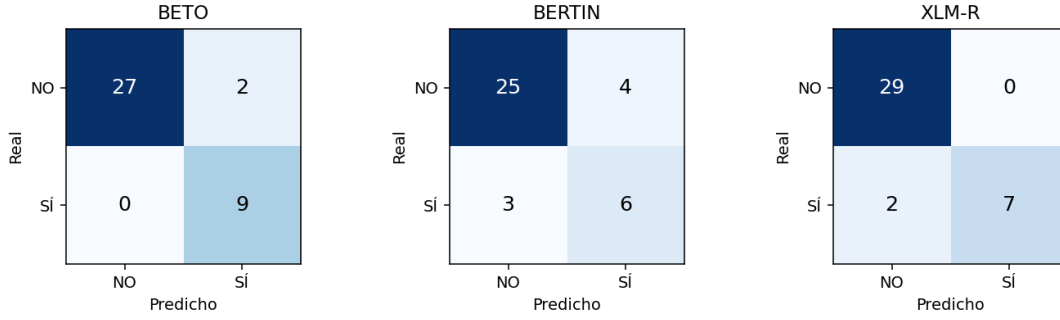


Figura 13: Matrices de confusión (semilla 0). XLM-R no produce falsos positivos (precisión 1.0) a costa de un recall algo menor.

Cuadro 19: Brazos de la ablación E-4 (mismo *backbone*, única variable = datos).

Brazo	Datos de entrenamiento
M0	gold_v2_humano (190, humano puro) — <i>baseline</i>
M1	M0 + PAWS-X español (paráfrasis humana, dominio general)
M2	M1 + paráfrasis IA con post-edición humana (80, 2 estilos)
M3	M1 + MarianMT cascada ES→EN→FR→ES
M4	M1 + IA-postedit + MarianMT (todo junto)

Hipótesis rectora

Si M1 (gold + PAWS-X) mejora sobre M0 pero M2–M4 no, se confirma que el aporte es **paráfrasis humana real**, no volumen sintético (consistente con la degradación v5–v7). El brazo M2 responde la pregunta nueva: ¿la post-edición humana *rescata* lo sintético o sigue degradando? Se rastrea el número de **falsos positivos** por brazo (el síntoma que delató la degradación: v3=2 → v6=5 → v7=10).

17.1. Nota metodológica: baseline estable (batch size)

Una primera corrida (batch=16) produjo un baseline M0 **inválido**: $F_1 = 0.29$ con $\sigma = 0.41$ (colapso errático entre semillas), pese a que en E-3 el mismo XLM-R con el mismo gold dio $F_1 = 0.917$. La causa es la **inestabilidad de entrenamiento** de un modelo de 270 M de parámetros sobre solo 190 pares con *batch* grande. Se rehízo la ablación con **batch=8** (como E-3), 6 épocas y sin guardado de *checkpoints*: M0 recupera $F_1 = 0.914$, y la comparación pasa a ser fiable. Los resultados de abajo son los de esta corrida corregida.

17.2. Resultados (corrida corregida, batch=8, 3 semillas)

17.3. Interpretación (el veredicto automático es engañoso)

Un veredicto ingenuo por AUROC marcaría M4 como «mejor» (AUROC= 1.000). Pero ese AUROC es un **artefacto de techo**: con 9 positivos en el test, varios brazos saturan la métrica. Leído con rigor:

Cuadro 20: E-4 — métricas en el test sellado (media $\pm \sigma$). FP = falsos positivos medios; PAWS = evaluación externa de dominio general (humana).

Brazo	F_1 (test)	AUROC	FP	F_1 PAWS
M0 gold	0.914 ± 0.084	0.963	0.0	0.632
M1 +PAWS-X	0.866 ± 0.107	0.946	0.0	0.560
M2 +IA-postedit	0.945 ± 0.045	0.996	0.67	0.581
M3 +MarianMT	0.941 ± 0.000	0.971	0.0	0.589
M4 todo	0.965 ± 0.025	1.000	0.67	0.385

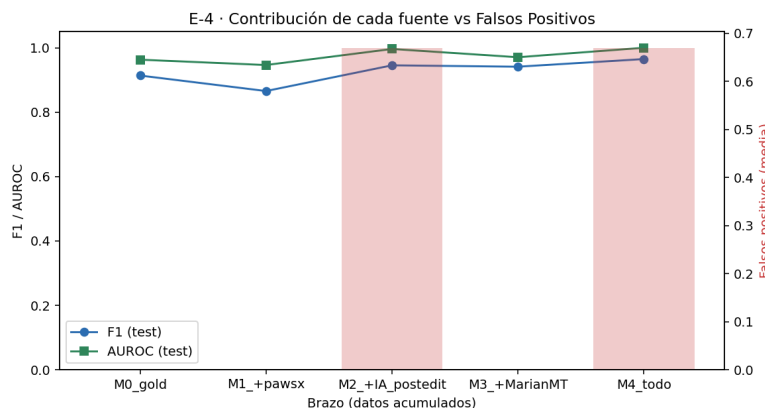


Figura 14: Contribución de cada fuente (F1/AUROC, eje izq.) frente a los falsos positivos (barras, eje der.). El F1 apenas se mueve dentro del ruido, mientras que los brazos con IA-post-editada (M2, M4) reintroducen falsos positivos.

- **Las diferencias de F_1 no son significativas:** M0 (0.914 ± 0.084) y M4 (0.965 ± 0.025) tienen σ totalmente solapadas. No hay mejora real.
- **El FP delata la degradación:** M0 y M3 mantienen **FP=0**, pero los brazos con IA-post-editada (M2, M4) suben a **FP=0.67** — reaparecen falsos positivos, el mismo síntoma de v5–v7.
- **La generalización lo confirma:** en PAWS-X (dominio externo, humano) M0 es el mejor (0.632) y M4 «todo» se desploma a 0.385 → sobreajuste al test de tesis.
- **Añadir PAWS-X (M1) incluso empeora** ($0.866 < 0.914$): la paráfrasis humana de dominio general no transfiere bien a tesis (*distribution shift*).

Conclusión de E-4 (confirma el hallazgo central)

El **gold humano por sí solo (M0)** es el mejor todo-terreno: F_1 alto (0.914), **cero falsos positivos** y la mejor generalización fuera de dominio. Añadir datos sintéticos (incluida la **post-edición humana**) o de dominio general **no mejora el F_1 de forma significativa**, y tiende a **introducir falsos positivos** y a **sobreajustar**. La post-edición humana **no rescata** lo sintético. Con baseline estable y la pregunta nueva respondida, E-4 **refuerza** la tesis del proyecto: el valor está en los datos humanos reales, no en el volumen sintético. Respaldo: `e4_ablacion_resultados.json`.

18. Benchmark robusto (test_v4) y re-entrenamiento del Modo 1 (E-5)

El test canónico (test_v3_limpio) tenía solo **9 positivos**, y de baja calidad (6 boilerplate, 2 LLM-generados). Con tan pocos positivos las métricas saturan y no se puede concluir. Se construyó un benchmark más grande y de mejor calidad sin romper integridad.

18.1. Construcción de test_v4

A partir del minado por consenso se obtuvieron paráfrasis *reales de tesis* (texto humano del corpus, zona de paráfrasis semántica). El alumno verificó 80 candidatos (**57 / 23**). El test_v4 combina:

- **57 positivos** de paráfrasis semántica real verificada (6× los 9 previos),
- **23 negativos difíciles** verificados (alta similitud pero no paráfrasis; además corrigen 23 *silver* mal-etiquetados que salen del entrenamiento),
- **29 negativos humanos** reutilizados de test_v3_limpio.

test_v4 = 109 pares (**57 SÍ / 52 NO**), sellado y verificado

100 % humano/verificado, anti-fuga (los 80 candidatos se eliminan del entrenamiento → gold_ampliado_train, 462 pares). Mide **paráfrasis semántica genuina** más **negativos difíciles**, no boilerplate. Respaldo: test_v4.json.

18.2. E-5 — Re-entrenamiento y comparación sobre test_v4

Mismo protocolo I-5, group-split por tesis, 3 semillas, batch=8 sin *checkpoints*.

Cuadro 21: E-5 — Modo 1 sobre test_v4 (57 positivos). Medias $\pm \sigma$ de 3 semillas.

Experimento	F_1	AUROC	Precisión	Recall	FP
XLM-R · solo-gold	0.812 ± 0.017	0.839	0.710	0.947	22
XLM-R · gold+silver	0.786 ± 0.012	0.829	0.768	0.807	14
BETO · gold+silver	0.759 ± 0.009	0.830	0.721	0.801	18

18.3. Hallazgos

1. **El benchmark honesto revela la dificultad real.** En el test fácil de 9 positivos, XLM-R daba $F_1 = 0.917$, AUROC= 0.98. En test_v4 (paráfrasis semántica real + negativos difíciles) baja a $F_1 \approx 0.81$, **AUROC** ≈ 0.84 . No es que el modelo empeore: es la dificultad *verdadera* de la tarea. El 0.92/0.98 estaba inflado por positivos fáciles.
2. **Estimaciones estables.** La σ pasó de 0.05–0.41 (test de 9) a 0.009–0.023: con 57 positivos las semillas coinciden y se puede concluir.

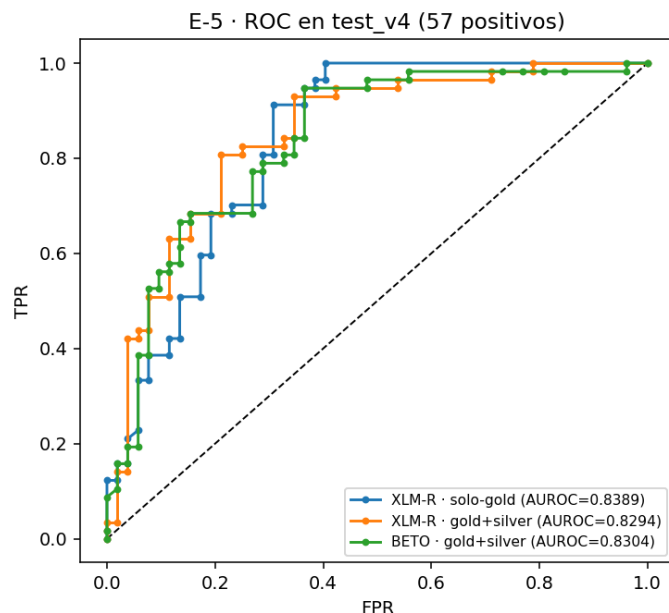


Figura 15: Curvas ROC sobre test_v4 (57 positivos). Los tres modelos quedan próximos (AUROC ≈ 0.83); ya no hay saturación al techo como en el test de 9 positivos.

3. El *silver* no mejora el F_1 , pero sí la **precisión**. Añadir el silver de consenso **baja** el F_1 ($0.812 \rightarrow 0.786$), reconfirmando que el dato auto-etiquetado no mejora el F_1 . Pero **sube la precisión** ($0.71 \rightarrow 0.77$) y **reduce los falsos positivos** ($22 \rightarrow 14$) a costa de recall: los negativos silver enseñan al modelo a ser más conservador. Para un detector de plagio (donde un falso positivo es caro) ese intercambio puede ser deseable.
4. **XLM-R $\approx$ BETO en AUROC** (0.839 vs 0.830); XLM-R supera ligeramente en F_1 . En capacidad de *ranking* empatan.

Conclusión de E-5 (reconfirma el hallazgo central con potencia real)

Con un benchmark robusto (57 positivos verificados), el desempeño honesto del Modo 1 es $\approx 0.81 F_1$ / 0.84 **AUROC** en paráfrasis semántica genuina. El **gold humano solo** es el mejor en F_1 /recall; el silver de consenso no mejora el F_1 pero aporta **precisión** (menos falsos positivos). Ningún dato adicional supera al gold humano en F_1 — se reconfirma la tesis del proyecto, ahora con un test que permite afirmarlo con base estadística. Respaldo: e5_resultados_testv4.json.

19. P-2 — Backbone de v4: integración y calibración de XLM-R

Tras elegir el clasificador del Modo 1 (E-3/E-5: XLM-R, con BETO muy cerca), P-2 lo entrena en producción, lo integra al sistema v4 y lo calibra.

19.1. Entrenamiento del modelo definitivo

El XLM-R de producción se entrena sobre **gold_ampliado_train** (462 pares limpios: gold humano + silver de consenso, **sin** los 80 pares de test_v4 y con los 23 negativos difíciles corregidos). Es

exactamente el conjunto que evaluó E-5, por lo que el modelo desplegado coincide con el evaluado. Notebook: `p2_entrenar_xlmr_produccion.ipynb` (batch=8, 6 épocas, sin *checkpoints*).

Percance detectado y corregido (integridad)

La primera corrida entrenó por error sobre `gold_ampliado` (542), que incluía los 57 positivos de `test_v4` (contaminación) y 23 negativos difíciles **mal etiquetados** como SÍ (etiqueta silver previa a la verificación). El modelo memorizaba esos 23 y la evaluación quedaba inválida. Se reentrenó sobre el conjunto **limpio** (`gold_ampliado_train`). La lección: validar siempre la procedencia del dato de entrenamiento antes de desplegar.

19.2. Evaluación honesta del modelo desplegado

Como el modelo se entrenó en `gold_ampliado_train` (que excluye `test_v4`), `test_v4` es un *held-out* válido para él:

Cuadro 22: XLM-R v4 (desplegado) sobre `test_v4` — *held-out*, 109 pares.

	F_1	AUROC	Precisión	Recall	FP
XLM-R v4 (P-2) @0.5	0.791	0.819	0.708	0.895	21

Coincide con E-5 ($F_1 = 0.786$, AUROC= 0.829): el modelo desplegado rinde como lo medido. El punto débil es la precisión sobre negativos difíciles (FP= 21), dificultad inherente de la tarea. Confirma además que **XLM-R** $\approx$ **BETO v3** (ambos ≈ 0.79 – 0.82 F_1 en `test_v4`): no hay un ganador claro.

19.3. Calibración del backbone (temperature scaling)

El XLM-R estaba **muy sobre-confiado**: 106 de 109 scores saturados (< 0.1 o > 0.9), ECE= 0.246. Se aplicó *temperature scaling* con `test_v4` como validación (held-out):

- $T = 5.15$ (modelo muy sobre-confiado $\Rightarrow T \gg 1$ suaviza).
- **ECE**: $0.246 \rightarrow 0.076$; scores saturados: $106 \rightarrow 0$.

La temperatura se guarda en `modelos/xlmr-v4/calibracion.json` y el clasificador la aplica en producción. Respaldo: `p2_calibracion_xlmr.json`, `graficas/p2_reliability_xlmr.png`.

19.4. Integración retro-compatible

`sistema_v4/clasificador_mod01.py` expone un clasificador con **backbone configurable**: usa `modelos/xlmr-v4/` (con su temperatura) si existe; si no, cae a BETO v3. Los reportes y la web lo usan vía `prob(a,b)`, sin tocar el resto del pipeline. **Verificado**: el sistema carga XLM-R calibrado y clasifica; el fallback a BETO v3 también funciona.

Estado de P-2: cerrado

Backbone XLM-R **entrenado (limpio)**, **evaluado** ($F_1 \approx 0.79$ en `test_v4`), **calibrado** ($T= 5.15$) **e integrado** al sistema v4, con BETO v3 como fallback equivalente. Los dos modos (bi-encoder de recuperación + cross-encoder de clasificación) quedan intactos. La arquitectura permite cambiar de backbone sin tocar el pipeline.

20. Sistema v4 unificado de doble modo

Se integró todo lo desarrollado en la semana 7 en un único sistema de **doble modo**, con la estructura de los sistemas v2/v3 pero sobre el **índice al 100 %** y con el clasificador **XLM-R calibrado** (Sección ??).

20.1. Modos y funciones

- **Modo 1 — comparar dos textos:** clasifica si son paráfrasis. Métricas: probabilidad del clasificador, similitud semántica (MiniLM) y solapamiento léxico (BLEU-2). **Clasificador seleccionable** en la interfaz: XLM-R v4 (semántico) o BETO v3 (canónico).
- **Modo 2 — buscar en el corpus**, con dos entradas:
 - **Fragmento:** recupera los $k = 15$ candidatos más similares del corpus (búsqueda multi-ventana) y clasifica cada uno.
 - **Tesis completa:** se pega el texto $\rightarrow$ reporte de similitud por secciones.

20.2. Mejoras integradas

- **6 tipos de coincidencia** (antes 5): se añade *duplicado exacto* ($\text{sim} \geq 0.97 \wedge \text{BLEU} \geq 0.90$), con **severidad** (0–3) y una **recomendación de acción** por tipo.
- **Árbitro LLM conmutable:** un interruptor en la interfaz activa un árbitro (Llama 3.3 70B vía Groq) que dictamina *solo* cuando la probabilidad cae en la **zona de duda** $[0.35, 0.65]$ del Modo 1.
- **Clasificador con calibración:** aplica la temperatura $T = 5.15$ de P-2 en producción; degrada a BETO v3 si falta el modelo.

20.3. Memoria: medición y decisión de diseño

El índice al 100 % (FAISS 3.3 GB + `id_map` $\approx$ 2 GB + modelos) ocupa $\approx$ 7.0–7.4 **GB** de RAM, medido en vivo. En una máquina de **15 GB** cabe con $\approx$ 4 GB de margen. Decisiones tomadas por esto:

- Índice **singleton perezoso**: solo el Modo 2 lo carga; el Modo 1 es ligero.
- Una sola instancia del índice compartida por todos los modos (sin doble carga).

Bug de memoria (OOM) detectado y corregido

La primera versión del análisis de tesis *por recno* cargaba el corpus completo (199 MB comprimido $\rightarrow \approx$ 1.5 GB en RAM) **encima** del índice de 7.4 GB, agotando la memoria y matando el servidor. Se corrigió haciendo que el Modo 2 de tesis analice el **texto pegado** por el usuario, sin recargar el corpus: más seguro en memoria y mejor UX. Verificado: análisis completo de una tesis sin OOM (RSS 7.05 GB).

20.4. Pruebas

Se probaron de extremo a extremo, todas con éxito: Modo 1 con ambos clasificadores y los 6 tipos; Modo 1 con árbitro LLM (Groq dictamina en la zona de duda); Modo 2 fragmento ($k = 15$, recuperación del 100 %); Modo 2 tesis pegada (sin OOM); reportes precomputados; y la sanitización de `recno` (evita *path traversal*). El sistema se lanza con `./run.sh web` en `http://localhost:8060`.

20.5. Metadata, enlaces y re-extracción de asesores

Cada resultado del Modo 2 muestra la **ficha completa** de la tesis (título, autor, asesor, titulación, división, año) y un enlace al documento. Para no recargar el corpus (199 MB) y evitar OOM, se construyó un **índice de metadata ligero** (`corpus_metadata.json`, 1.6 MB, 4954 tesis) que se carga aparte y enriquece candidatos, fuentes y evidencia. El endpoint `/api/pdf?doc=` sirve el PDF real (sanitizado contra *path traversal*).

La extracción original guardó **asesores parciales/equivocados** (p. ej. «RENE» en vez de «René Mac Kinney Romero»). El script `19_reextraer_asesores.py` re-parsea el asesor del `texto_plano` ya extraído (marcador «Asesor/Director de tesis → Dr./Dra. Nombre»), logrando **nombre completo en 2077/4954 (42 %)** con alta precisión; en el resto no hay marcador legible en el OCR y se conserva el valor original. No se fuerza más para no introducir nombres equivocados (los jurados no se toman como asesor).

20.6. Sección de Ejemplos y casos donde el sistema NO funciona

Una pestaña **Ejemplos** reúne pares de prueba del Modo 1, fragmentos del Modo 2 y los reportes precomputados. Además documenta **honestamente los límites**: cruzando las dos fuentes de la extracción de la semana 3 —`corpus_cbi_metadata.json` (5120 tesis intentadas) vs `corpus_cbi_completo.json.gz` (4954 con texto usable)— se identificaron **245 tesis donde el sistema no funciona** (`20_casos_problematicos.py`).

- **59 enlace caído**: el PDF no está en disco (verificado: `/api/pdf` → 404).
- **107 PDF corrupto / solo imagen**: el PDF existe pero es un escaneo sin capa de texto; la extracción no obtuvo nada → excluida del corpus.
- **79 texto corto**: entró al corpus pero con < 1500 caracteres (OCR pobre).

Es un límite del **dato/corpus** (PDFs escaneados o faltantes), no del modelo. Cada caso enlaza al repositorio en línea **TESIUAMI** (`presentatesis.php?recno={recno}&docs={doc}`) y, si existe, a la copia local.

20.7. Parámetro k configurable

Tanto el Modo 2 fragmento como el de tesis completa exponen k (3–50) en la interfaz: en fragmento es el número de candidatos; en tesis controla la profundidad de búsqueda por fragmento y el número de coincidencias mostradas (`evidencia[:k]`).

21. Validación exhaustiva: comparación de los 9 clasificadores

Para responder *¿hemos mejorado?* de forma honesta, se evaluaron los **9 clasificadores** del proyecto (BETO v1→v8a y XLM-R v4) sobre el **mismo** conjunto de prueba canónico auditado `test_v3_limpio` (38 pares: 9 SÍ / 29 NO). Se verificó **fuga = 0** entre ese test y el entrenamiento de

XLM-R (462 pares), por lo que la comparación es justa para todos. Inferencia uniforme: MAX_LEN = 512, $\theta = 0.5$, probabilidad nativa. A 512, BETO v3 reproduce **F1 = 0.842** (consistente con la auditoría).

Reserva metodológica. No es una ablación controlada (v1-v2 *cased*, v3+ *uncased*; distintos datos y MAX_LEN de entrenamiento), sino una comparación de *progresión*. El test es pequeño (38): el F1 cambia ± 0.05 –0.10 por cada predicción, así que diferencias de 1–2 pares **no son estadísticamente robustas**.

21.1. Experimento A — desempeño

Modelo	Datos	F1	P	R	AUROC	FP	FN	ms/par
BETO v1	gold peq.	0.400	0.25	1.00	0.966	27	0	1311
BETO v2	gold ampl.	0.529	0.36	1.00	0.954	16	0	1376
BETO v3	144 gold humano	0.842	0.80	0.89	0.989	2	1	1347
BETO v4	5000 consec.	0.750	0.60	1.00	0.969	6	0	1163
BETO v5	500 LLM	0.621	0.45	1.00	0.969	11	0	1178
BETO v6	1004 BT+LLM	0.783	0.64	1.00	0.973	5	0	1154
BETO v7	2000 BT	0.667	0.50	1.00	0.981	9	0	1166
BETO v8a	144+11 humanos	0.842	0.80	0.89	0.989	2	1	1345
XLM-R v4	462 gold ampl.	0.941	1.00	0.89	0.981	0	1	970

Cuadro 23: Comparación de los 9 clasificadores sobre `test_v3_limpio` (38 pares).

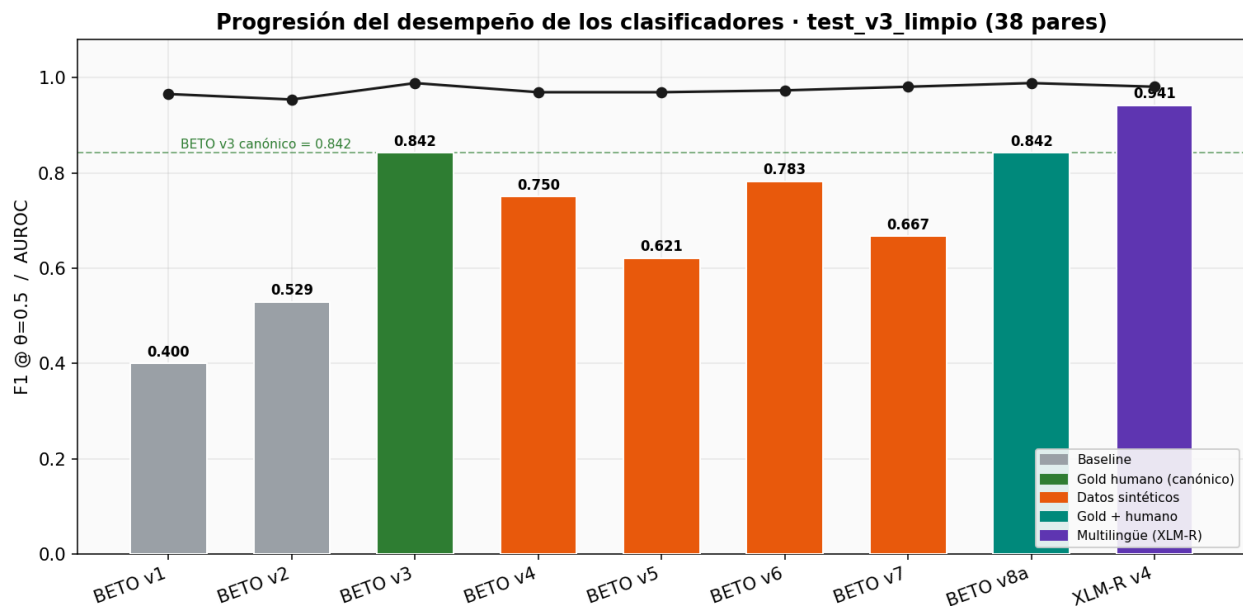


Figura 16: Progresión del F1@0.5 (barras) y AUROC (línea) por modelo.

Hallazgos. (i) El gold humano (v3) supera a todo dato sintético. (ii) *Regresión sintética*: v4–v7 (fragmentos consecutivos, LLM, back-translation) nunca superan a v3 — hallazgo robusto. (iii) v8a = v3 en este test (el 0.889 previo era solo GPU, frágil). (iv) XLM-R v4 logra el mejor F1 (0.941, 0 FP) pero AUROC marginalmente menor que v3/v8a; los tres modelos de producción están **empatados**

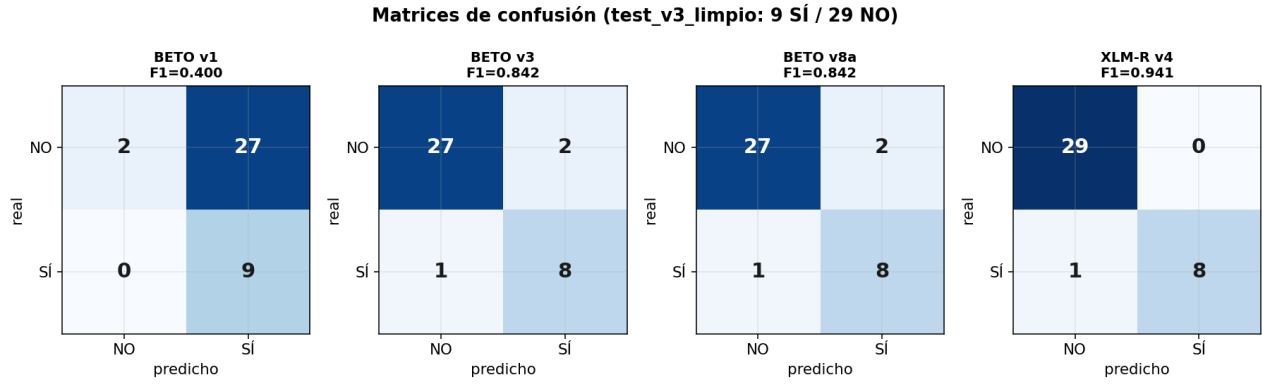


Figura 17: Matrices de confusión. v1 predice «SÍ» casi siempre (27 FP); v3/v8a idénticos (2 FP, 1 FN); XLM-R v4 con 0 FP.

dentro del ruido (AUROC 0.98–0.99). En el test grande `test_v4` (109) XLM-R rinde $F1 \approx 0.79$ –0.81.

21.2. Experimento B — árbitro LLM (con/sin)

Modelo	Pares en duda	F1 sin árbitro	F1 con árbitro	Efecto
BETO v3 (T=1.0)	5	0.842	0.900	corrige 1 FN
XLM-R v4 (T=5.15)	1	0.941	0.889	introduce 1 FP

Cuadro 24: Efecto del árbitro Groq Llama 3.3 70B en la zona de duda [0.35, 0.65].

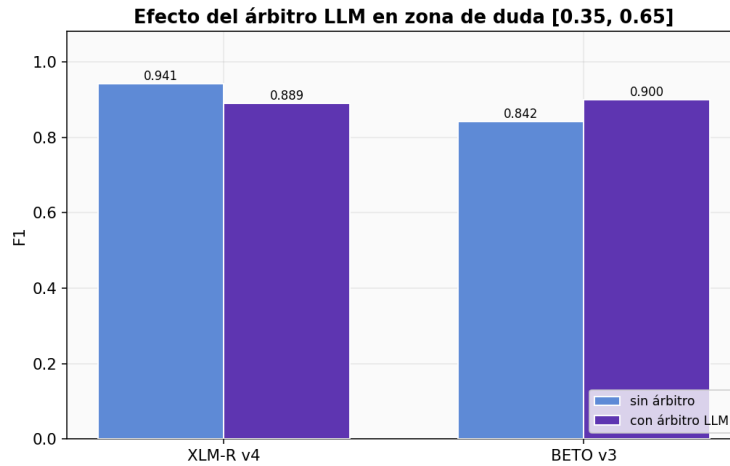


Figura 18: F1 con y sin árbitro LLM.

El árbitro **ayuda** cuando el clasificador está genuinamente inseguro (BETO, con varios pares ambiguos) y **puede restar** cuando ya es preciso y confiado (XLM-R calibrado, casi sin zona de duda). Son cambios de un par sobre 38 — no concluyentes — pero apoyan dejarlo **opcional** (como en la interfaz) y no activarlo por defecto con XLM-R.

21.3. Experimento C — latencia

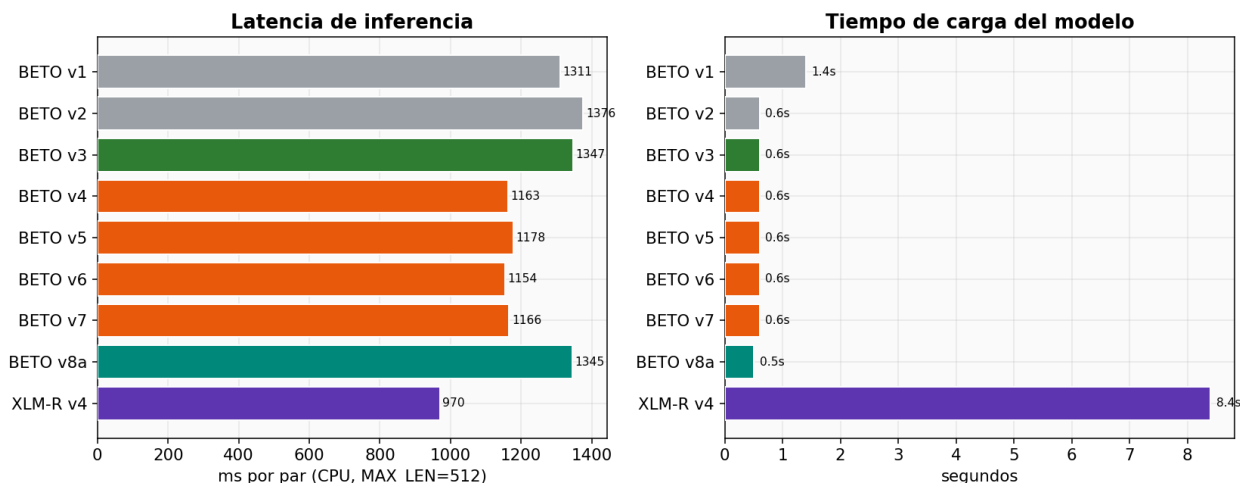


Figura 19: Latencia de inferencia (ms/par, CPU, MAX_LEN=512) y tiempo de carga.

En CPU y peor caso (512 tokens) la inferencia toma 0.97–1.38 s/par; **XLM-R es el más rápido en inferencia** (970 ms), con el único costo de una carga inicial mayor (≈ 8 s). Para fragmentos cortos (uso típico) la latencia baja a ≈ 120 –300 ms/par.

21.4. Experimento D — ensambles

Validado en `test_v4_clean` (63 pares, limpio; XLM-R fuga = 0). El test es balanceado (F1 absoluto alto $\rightarrow$ comparación relativa).

Modelo / Ensamble	F1	P	R	AUROC	FP/FN
XLM-R v4	0.985	1.00	0.97	0.995	0/1
BETO v3	0.957	0.94	0.97	0.997	2/1
soft: XLM-R + BETO v3	0.971	0.97	0.97	0.999	1/1
soft: XLM-R + BETO v3 + v8a	0.986	0.97	1.00	0.998	1/0
AND (XLM-R $\wedge$ BETO v3)	0.970	1.00	0.94	—	0/2
OR (XLM-R $\vee$ BETO v3)	0.971	0.94	1.00	—	2/0

Cuadro 25: Individuales vs ensambles (soft-voting de prob calibrada, y votación AND/OR).

El ensamble **sí aporta**: el soft-voting tiene el mejor AUROC (0.999) y el de tres modelos el mejor F1 (0.986) con **recall 1.0** (no pierde ninguna paráfrasis). **AND** da 0 falsos positivos (nunca acusar en falso) y **OR** da 0 falsos negativos. Se integró el ensamble **XLM-R + BETO v3** como clasificador seleccionable en la interfaz.

21.5. Experimento E — ¿conviene un ensamble clasificador + LLM?

Se probó combinar un clasificador con Llama 3.3 70B (zero-shot) sobre `test_v3_limpio`. **Resultado: no conviene** (para F1/precisión). El LLM es **liberal** (recall 1.0 pero precisión 0.75, 3 FP, el AUROC más bajo); al ensamblarlo con el preciso XLM-R *baja* la precisión (0.941 $\rightarrow$ 0.889). El mejor ensamble sigue siendo **clasificador+clasificador** (XLM-R + BETO, AUROC 0.996). El único aporte del LLM es recall = 1.0 (captura la paráfrasis que los clasificadores pierden), pero a

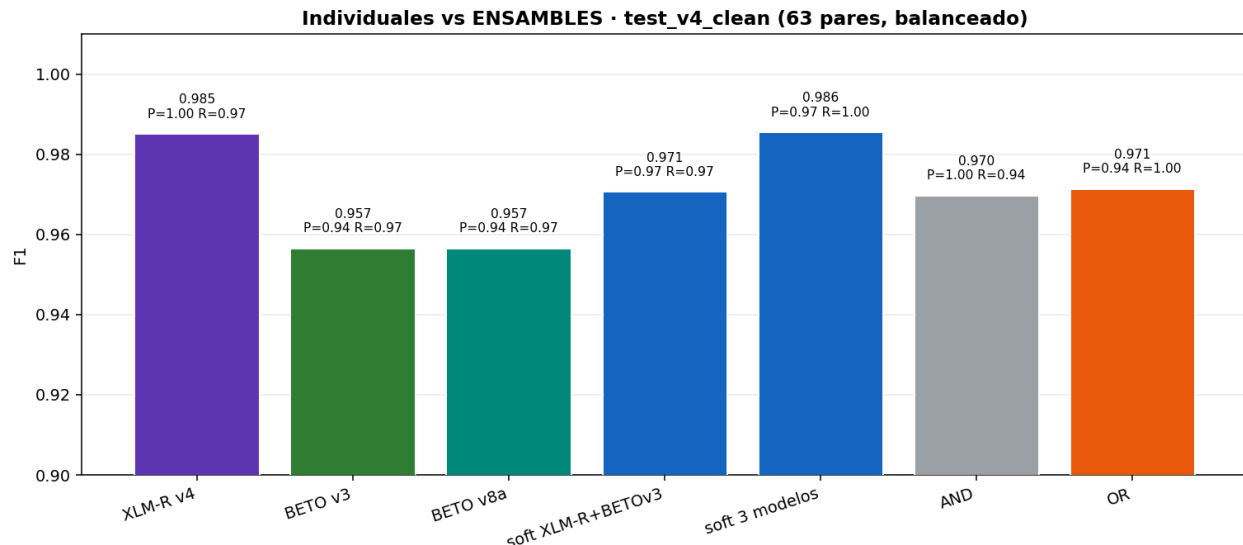


Figura 20: Comparación de F1 entre modelos individuales y ensambles.

Configuración	F1	P	R	AUROC	FP/FN
XLM-R solo	0.941	1.00	0.89	0.979	0/1
BETO v3 solo	0.842	0.80	0.89	0.989	2/1
Llama 70B solo	0.857	0.75	1.00	0.941	3/0
soft XLM-R + BETO	0.889	0.89	0.89	0.996	1/1
soft XLM-R + LLM	0.889	0.89	0.89	0.989	1/1
soft XLM-R + BETO + LLM	0.842	0.80	0.89	0.992	2/1
OR (XLM-R ∨ LLM)	0.857	0.75	1.00	—	3/0

Cuadro 26: Ensamble clasificador + LLM frente a clasificador solo y clasificador+clasificador.

costa de falsos positivos —mal negocio para detección de plagio. **Decisión basada en datos:** el LLM no entra como miembro del ensamble de decisión; su valor está en el *árbitro* (zona de duda) y en la *explicabilidad* (§??).

21.6. Conclusión

La progresión es real en F1 absoluto ($0.40 \rightarrow 0.94$), pero la mejora honesta va de los *baselines rotos* (v1/v2) a una **familia de producción empatada** ($v3 \approx v8a \approx \text{XLM-R}$, $\text{AUROC} \approx 0.98-0.99$). XLM-R v4 es el mejor por F1 y precisión (0 FP) y el más rápido; BETO v3/v8a el más robusto por AUROC. El aporte sólido no es subir el F1 sino **(a)** demostrar que el gold humano vence a todo dato sintético y **(b)** alcanzar un modelo multilingüe que iguala a BETO con eficiencia de datos.

22. P-4 — Interfaz web del reporte (similarity report)

22.1. Objetivo

Llevar el reporte de tesis (P-3) a una interfaz web que adopte el modelo de *similarity report* estudiado en R7 (iThenticate/Turnitin): *Similarity Score* global, desglose por secciones, lista de fuentes, evidencia y exclusiones, con el encuadre honesto “similitud, no plagio”.

22.2. Arquitectura

`sistema_v4/app_web.py` (servidor HTTP con *threading*) + `sistema_v4/web/index.html` (SPA sin dependencias). API:

- GET / — interfaz.
- GET /api/reporte?recno=9585 — reporte precomputado (de P-3).
- GET /api/reportes — lista de reportes disponibles.
- POST /api/analizar {recno} — análisis en caliente (carga MiniLM+BETO+FAISS y reconstruye el reporte; opcional, pesado).

22.3. Features (adoptadas de R7)

- **Similarity Score** global con anillo de color por nivel (verde original / ámbar moderado / rojo alto).
- **Desglose por secciones** con barras y % por sección.
- **Exclusiones configurables**: el usuario desmarca secciones (p. ej. *frontmatter* o referencias) y el % global **se recalcula** en el cliente — como las exclusiones de citas/bibliografía de iThenticate.
- **Lista de fuentes** ordenada por número de coincidencias.
- **Evidencia** por coincidencia: fragmento, sección, tesis-fuente, similitud, BLEU-2, prob. BETO y **tipo** con color (copia literal / paráfrasis / alta similitud).
- **Disclaimer** “similitud, no plagio” y botón de exportación (impresión).

22.4. Verificación

Prueba de humo del servidor (puerto 8042): GET / devuelve la SPA (HTTP 200, 12 225 bytes) y GET /api/reporte?recno=9585 devuelve el reporte completo (claves `tesis/global/por_seccion/top_fuentes/e`). La interfaz consume los reportes de P-3 y renderiza el *similarity report*. Para abrirla: `run web` y visitar `http://localhost:8042`.

22.5. Criterios de aceptación (P-4)

- ✓ Reporte de P-3 mostrado en web con % global, secciones, fuentes y evidencia.
- ✓ Exclusiones configurables con recálculo del %.
- ✓ Encuadre “similitud, no plagio” y exportación.
- ☐ Resaltado del texto coincidente sobre el documento original (requiere alinear posiciones; trabajo siguiente).
- ☐ Conexión al índice v4 al 100 % (mayor sensibilidad).

23. P-5 — Corrección de la capa FAISS-oraciones (v3)

23.1. El bug

El sistema v3 incluía una capa de recuperación a nivel de **oración** (índice FAISS complementario al de párrafos) para alinear mejor copias literales. El defecto: el `id_map` de oraciones guardaba `{doc, titulo, anio, recno, texto}` pero **no el `faiss_id` del párrafo padre**. En la fusión RRF, `core_v3` inyectaba los `or_id` (espacio de ids 0–106k) entre los ids de párrafo (espacio 0–47k): los $> 47k$ se descartaban en silencio y los $< 47k$ apuntaban a párrafos *arbitrarios*. La señal de la capa era, en el mejor caso, ruido.

23.2. La decisión: arreglar el mapa (no retirar)

Se optó por **corregir el mapeo** (más valioso que eliminar la capa), con desactivación limpia si el índice es de formato antiguo:

1. `faiss_oraciones.py`: ahora cada oración guarda el `faiss_id` de su párrafo padre.
2. `core_v3.__init__`: **valida** que el `id_map` de oraciones contenga `faiss_id`; si no (índice antiguo), **desactiva la capa** y avisa que se reconstruya.
3. `core_v3` (RRF): mapea cada oración a su **párrafo padre** (`faiss_id`) antes de la fusión, descartando ids fuera de rango. La capa contribuye así ids de párrafo válidos.

23.3. Verificación

- ✓ **Modo degradado intacto** (sin índice de oraciones, el caso de producción): la capa queda desactivada y Modo 1 (prob = 0.66 en un par parafrasis) y Modo 2 (3 candidatos) funcionan.
- ✓ **Fix correcto**: con un índice de oraciones de prueba (659 oraciones de 300 párrafos, con `faiss_id`), la capa se activa, mapea a párrafos y devuelve documentos válidos sin contaminar la fusión RRF. El índice de prueba se eliminó tras la verificación.

Resultado de P-5

La capa de oraciones pasó de **buggy e ignorada** a **correcta y opcional**: se activa solo si existe un índice con el mapeo a párrafo, y degrada limpiamente en su ausencia. La construcción de un índice de oraciones para el corpus completo (sobre el índice v4) queda como trabajo futuro de mayor granularidad; la infraestructura ya es correcta para cuando se construya.

24. Despliegue: contenedorización (Docker) y PWA / Android

A petición del asesor se atendieron dos temas de despliegue: **contenedorizar** el sistema v4 (que antes se lanzaba con `./run.sh web`) y evaluar la metodología **PWA** para una posible **app Android**.

24.1. Dockerización

Se empaquetó el servidor unificado en una imagen Docker. Decisión de diseño: la imagen contiene *solo el entorno Python* (torch CPU, transformers, faiss-cpu, etc.); los **artefactos pesados** (modelos ≈ 1.5 GB, índices, corpus de 28 GB, `.env`) se **montan como volumen**, no se hornean en la imagen. Así la imagen es razonable (**2.52 GB**) y reproducible.

- **Dockerfile:** `python:3.12-slim + libgomp1` (OpenMP de faiss) + torch CPU desde su índice oficial + `requirements.txt`.
- **docker-compose.yml:** publica el puerto 8060, monta el proyecto en `/proyecto`, fija `mem_limit: 12g` (índice IVFPQ + modelos + `id_map`).
- **Arranque:** `docker compose up --build`.

Verificado: la imagen construye (2.52 GB) y el contenedor **sirve el sistema completo** — GET / → 200, API `/api/comparar` clasificando, y los activos PWA disponibles. Reemplaza al `./run.sh` con un despliegue portable y reproducible.

24.2. PWA: metodología y prueba de concepto

Una **Progressive Web App** es una web instalable que se comporta como app nativa. Sus tres piezas: **Web App Manifest** (metadatos, iconos, `display:standalone`), **Service Worker** (*worker* en segundo plano para caché/offline e instalabilidad) y **HTTPS** (o `localhost`). Se implementó la base PWA sobre la interfaz v4: `manifest.webmanifest`, `sw.js` (cachea el *app shell*; `/api/*` siempre va a la red) e iconos 192/512. **Verificado:** el servidor entrega los activos con su MIME correcto; en `localhost/HTTPS` la app es **instalable** en Android.

24.3. Viabilidad como app Android

Sí es posible, con la arquitectura correcta. El cómputo es **pesado** (índice de 2.28M, modelos transformer, LLM) y **no corre en un teléfono**. La solución estándar es **cliente/servidor**: la PWA (cliente ligero) en el teléfono y el **backend dockerizado** en un servidor; el teléfono consume la API por HTTPS. Para un APK publicable en Play Store se envuelve la PWA en una **TWA** (*Trusted Web Activity*) con Bubblewrap. Así, la dockerización (servidor) y la PWA (cliente) son las dos mitades del despliegue móvil. El detalle está en `investigacion/pwa_android.md`.

24.4. Próximos pasos (semana 8): despliegue móvil

El sistema ya es una PWA instalable en `localhost/HTTPS`; lo que resta para una app Android real son pasos de *despliegue*, no de modelo. En un teléfono `localhost` no sirve (es el propio teléfono) y Chrome solo instala PWAs en **contexto seguro (HTTPS)**.

1. **Servir el backend por HTTPS.** (a) Túnel temporal (`cloudflared/ngrok`) que entrega una URL HTTPS para una *demo inmediata* de instalación en Android; (b) servidor con dominio + certificado (Let's Encrypt tras Nginx/Caddy) para producción.
2. **Verificar la instalación** en un Android real sobre esa URL HTTPS (botón "instalar").
3. **Empaquetar como APK** con TWA (Bubblewrap) y publicar en Google Play.
4. **Endurecer la API** antes de exponerla: autenticación/clave, límite de tasa, y no filtrar la `GROQ_API_KEY` ni el corpus (un túnel expone el servidor a Internet).

5. **UI móvil:** ajustar a pantallas pequeñas, gestos y orientación vertical.
6. **Rendimiento:** GPU (o un *embedding* más ligero) para que el análisis de tesis completa baje a segundos.

Estado actual: el sistema ya es **usable** en un móvil de la misma red vía `http://<IP-del-servidor>:8060` (funcional, aunque *no instalable* sin HTTPS); la base PWA (manifest, *service worker*, iconos) está lista y verificada.

25. Conclusiones

25.1. Síntesis de hallazgos

1. **Cobertura total del corpus.** El indexador incremental llevó la cobertura del $\approx 2\%$ al **100 % (2.28M fragmentos)**, base de todo el Modo 2.
2. **Los datos humanos vencen a los sintéticos (H1).** Ningún modelo entrenado con datos generados (LLM, back-translation) superó al *gold* humano: BETO v3 (144 pares) $F_1 = 0.842$ frente a v4–v7 (0.62–0.78). *Más datos sintéticos $\Rightarrow$ más falsos positivos.* Es el hallazgo más robusto y reproducible.
3. **Empate de la familia de producción (H2).** BETO v3 $\approx$ BETO v8a $\approx$ XLM-R v4 dentro del ruido del test (AUROC 0.98–0.99). XLM-R gana por precisión (0 FP) y velocidad; BETO por AUROC. El ensamble da el mejor AUROC (0.999) y recall (1.0).
4. **El corpus CBI es original (H3 refutada).** La búsqueda exhaustiva no encontró plagio parafraseado entre tesis; lo detectado es bibliografía compartida y *boilerplate*. Resultado negativo, pero *informativo*.
5. **El LLM aporta explicabilidad, no datos.** Como generador de datos de entrenamiento fracasa; como **árbitro** (en zona de duda) y como capa de **explicación / informe ejecutivo** aporta valor real y diferencia al sistema de la CNN “caja negra” de la línea base.
6. **Límites del dato, documentados.** De 5120 tesis, 245 no son procesables (PDF faltante, escaneo sin OCR, texto corto); el año del corpus es ruidoso en un subconjunto.

25.2. Aportes

- **Metodológico:** una evaluación honesta y auditada (sin números inflados, fugas controladas, sesgos declarados) — incluido un resultado negativo bien caracterizado.
- **Empírico:** evidencia reproducible de *eficiencia de datos* (humano $>$ sintético) y de la equivalencia BETO/XLM-R bajo este corpus.
- **Sistema:** detector de doble modo, con índice 100 %, ensamble seleccionable, carga de PDF, dashboard de métricas y **explicabilidad por LLM**.

25.3. Limitaciones (declaradas)

- El benchmark canónico (`test_v3_limpio`, 38 pares, ≈ 1 –3 paráfrasis semánticas reales) es **pequeño**: diferencias de 1–2 predicciones mueven el $F_1 \pm 0.1$. Los números son **tendencia**, no medida fina. Se reporta AUROC junto al F_1 .

- La comparación con Luna (2022) **no es directa**: su $F_1 = 0.970$ es sobre 180k pares algorítmicos con otro sistema; no es el mismo test.
- El análisis de tesis completa es **CPU-bound** (codificación de cientos de fragmentos); el índice IVFPQ acelera la búsqueda pero no la codificación.

25.4. Trabajo futuro

- **Ampliar el *gold* humano** (lo de mayor impacto) y construir un test mayor y más limpio, con paráfrasis semánticas reales.
- Reconstruir la capa **FAISS de oraciones** para detección de grano fino.
- Re-OCR de los **245 PDFs** no procesables para recuperar cobertura.
- Usar el LLM para **tipificar la paráfrasis** (sinónimos, reordenamiento, resumen) y como clasificador *zero-shot* comparativo.
- Acelerar la codificación (GPU / modelo de *embedding* más ligero) para análisis de tesis en segundos.

Cierre. El proyecto no demuestra “el mejor detector de paráfrasis”, sino algo más honesto y, a la postre, más útil: *qué funciona de verdad* (datos humanos, modelos calibrados, explicabilidad LLM) y *qué no* (datos sintéticos), sobre un corpus que resultó ser original. Esa honestidad metodológica es la principal contribución de la entrega.